Guía de estudio: la base de datos del sistema escolar

Oct 13, 2026 · @juan carlos bodoque III

1. Cómo usar esta guía y mapa general

Tu base de datos no solo guarda información: también decide qué está permitido y qué no. Entender eso es la clave de toda la defensa.

Esta guía va de lo simple a lo complejo. Primero los fundamentos, luego las tablas, las vistas, los procedimientos almacenados y los triggers. Al final verás tres flujos completos de punta a punta y las preguntas con respuesta larga. Si tienes dos o tres días, una sección por sesión funciona bien.

Qué contiene el script

El archivo `BaseDeDatos_Completa.sql` tiene cinco tipos de objetos. Cada uno resuelve un problema distinto.

Pieza	Cantidad	Qué problema resuelve	Ejemplo real
Tablas	30	Guardar los datos de forma ordenada y protegida	`Empleados`, `UnidadesActivo`, `Prestamos`
Funciones	6	Cálculos reutilizables que devuelven un valor o una tabla	`fn_DiasHabiles`, `fn_PeriodoCerrado`
Vistas	27	Consultas guardadas con nombre, listas para mostrar en pantalla	`vw_Prestamos`, `vw_StockConsumibles`
Procedimientos almacenados	34	Operaciones completas con reglas de negocio	`sp_RegistrarPrestamo`, `sp_GenerarPlanilla`

Pieza	Cantidad	Qué problema resuelve	Ejemplo real
Triggers	20	Código que se dispara solo cuando cambia una tabla	`trg_Aud_Empleados`, `trg_Auditoria_Inmutable`

De los 20 triggers, 17 son de registro de actividad (uno por tabla auditada), uno protege el registro para que nadie lo borre, y dos llevan el historial del inventario.

Los siete grupos de tablas

Las 30 tablas se entienden mejor por grupos. Cada grupo corresponde a una parte del sistema que ves en el menú.

Grupo	Tablas	Para qué sirve
Personal y acceso	`Roles`, `Usuarios`, `Cargos`, `EstadosEmpleado`, `Empleados`, `EmpleadoFotos`, `EmpleadoMaterias`	Quién trabaja en la institución y quién puede entrar al sistema
Académico	`Grados`, `Secciones`, `Materias`, `AsignacionesDocente`	Qué docente da qué materia en qué grupo y cuántas horas
Planillas	`Planillas`, `PagosPlanilla`, `PeriodosPlanilla`, `PeriodosPlanillaHistorial`, `Ausencias`	Salarios, pagos mensuales, cierre de mes y descuentos por faltas
Activos	`Categorias`, `Estados`, `GestionActivos`, `UnidadesActivo`, `ReportesDanio`, `Prestamos`, `MantenimientosActivo`, `AuditoriaUnidades`, `AuditoriaActivos`	El inventario físico, su estado, sus préstamos y su mantenimiento
Salones	`Salones`	Dónde está cada cosa; incluye la bodega
Consumibles	`Consumibles`, `MovimientosConsumible`	Papel, marcadores y similares, con su kardex

Grupo	Tablas	Para qué sirve
Sistema	`ConfiguracionEmpresa`, `Auditoria`	Datos de la institución y registro de quién hizo qué

La idea que ordena todo

Hay dos lugares donde puede vivir una regla: en la pantalla (C#) o en la base de datos. Tu proyecto pone las reglas importantes en la base. La razón es simple: la pantalla puede cambiar, pero los datos son lo que no se puede dañar.

Un ejemplo concreto. El numérico de "Asignar equipo" ya no deja escribir más de lo disponible; eso es una comodidad de la pantalla. Aunque esa comodidad fallara, `sp_AsignarUnidades` vuelve a contar las unidades disponibles y rechaza el pedido con el error 50514. La pantalla ayuda al usuario; la base garantiza la verdad.

Hay un matiz honesto que conviene que sepas. Las restricciones de las tablas (`CHECK`, `UNIQUE`, llaves foráneas) protegen siempre, incluso contra un `UPDATE` directo. Las reglas más elaboradas, como "una baja necesita motivo", viven en los procedimientos; un `UPDATE` escrito a mano sobre la tabla se las salta. Por eso el sistema siempre pasa por los procedimientos.

2. Fundamentos que necesitas dominar

Una base de datos relacional es un conjunto de tablas conectadas por llaves, y casi todo lo que hace tu script se explica con siete ideas. Si las entiendes bien, el resto se deduce solo.

2.1 Tabla, fila y columna

Una tabla guarda un tipo de cosa: `Empleados` guarda personas, `Prestamos` guarda préstamos. Cada fila es un caso concreto (un empleado, un préstamo) y cada columna es una característica (nombre, fecha límite). El tipo de la columna ya es una primera protección: el dinero va en `DECIMAL(10,2)` porque los decimales exactos no se redondean mal como los `float`, y una fecha sin hora va en `DATE`.

2.2 Llave primaria: la identidad de cada fila

La llave primaria (`PRIMARY KEY`) es la columna que identifica una fila sin ambigüedad. Casi todas tus tablas usan `IdAlgo INT IDENTITY(1,1)`: el motor numera solo, 1, 2, 3, y nunca repite. Eso permite que otras tablas apunten a esa fila por su número en vez de

copiar el nombre.

Algunas llaves usan dos columnas. `EmpleadoMaterias` tiene `PRIMARY KEY (IdEmpleado, IdMateria)`: puede haber muchas filas por empleado y muchas por materia, pero la pareja no se repite. Un docente no puede tener "Matemática" dos veces.

2.3 Llave foránea: cómo se conectan las tablas

Una llave foránea (`FOREIGN KEY`) es una columna que apunta a la llave primaria de otra tabla. `Empleados.IdCargo` apunta a `Cargos.IdCargo`. La consecuencia importante es la integridad referencial: el motor no te deja guardar un empleado con un cargo que no existe, ni borrar un cargo que aún usa alguien.

Hay dos tipos de relación que debes saber nombrar:

- **Uno a muchos:** un cargo tiene muchos empleados; un empleado tiene muchos pagos. Es la más común y se resuelve con una llave foránea en el lado "muchos".
- **Muchos a muchos:** un docente da varias materias y una materia la dan varios docentes. Se resuelve con una tabla intermedia, `EmpleadoMaterias`, que solo guarda parejas.

Algunas llaves llevan `ON DELETE CASCADE`: si se borra el equipo del catálogo, se borran sus unidades. Es una decisión de diseño consciente: una unidad no tiene sentido sin su equipo.

2.4 Restricciones: las reglas que guarda la tabla

Las restricciones son las reglas más baratas y más fuertes, porque el motor las aplica siempre. Tu script usa cinco tipos.

Restricción	Qué dice	Ejemplo de tu script
`NOT NULL`	La columna no puede quedar vacía	`Usuarios.Clave`
`DEFAULT`	Valor que se pone si nadie indica otro	`Prestamos.Estado` empieza en 'Activo'
`UNIQUE`	Ningún valor se repite	`Empleados.Nombre`, `Usuarios.Usuario`
`CHECK`	Una condición que cada fila debe cumplir	`Salones.Capacidad` entre 1 y 500

Restricción	Qué dice	Ejemplo de tu script
`FOREIGN KEY`	El valor debe existir en otra tabla	`Prestamos.IdUnidad`

El `CHECK` más interesante es el de `PagosPlanilla`: obliga a que el salario base sea mayor que cero y a que el descuento no supere base más bono. Un pago imposible no puede existir ni por error.

2.5 Índices únicos filtrados: reglas con condición

A veces la unicidad solo aplica a una parte de las filas. "Un empleado tiene un solo pago por mes" es falso si cuentas los pagos anulados, porque se puede anular y volver a registrar. La solución es un índice único con `WHERE`:

```
CREATE UNIQUE INDEX UX_PagosPlanilla_Empleado_Periodo
    ON dbo.PagosPlanilla (IdEmpleado, Periodo) WHERE ([Anulado] = 0);
```

Los pagos anulados quedan fuera del índice, así que no estorban. El mismo truco impide dos préstamos activos de la misma unidad (`UX_Prestamos_UnidadActiva`) y dos reportes de daño abiertos (`UX_ReportesDanio_UnidadAbierta`).

2.6 Columnas calculadas

`PagosPlanilla.SalarioNeto` se declara como `AS (SalarioBase + Bono - Descuento) PERSISTED`. Nadie la escribe: el motor la calcula y la guarda. Así el neto nunca puede quedar desincronizado con sus componentes.

2.7 Normalización: no repetir información

Normalizar es guardar cada dato en un solo lugar. Por eso existe `Cargos` (el texto "Coordinador Académico" se guarda una vez, no en cada empleado) y por eso el inventario se divide en dos niveles: `GestionActivos` es el catálogo ("Pupitre") y `UnidadesActivo` son las piezas físicas, cada una con su código y estado. Si el estado viviéra en el catálogo, todos los pupitres tendrían el mismo estado, que fue el error del diseño original.

2.8 Transacciones: todo o nada

Una transacción agrupa varias instrucciones para que se cumplan todas o ninguna. Imagina una devolución: hay que cerrar el préstamo y cambiar el estado de la unidad. Si se cierra el préstamo y luego algo falla, la unidad quedaría con un estado viejo. Con `BEGIN TRANSACTION ... COMMIT` o `ROLLBACK`, esa mitad nunca se queda guardada.

Tus procedimientos además usan `SET XACT_ABORT ON` (si hay un error, la transacción se cancela entera) y bloqueos como `UPDLOCK, HOLDLOCK`. Esos bloqueos evitan que dos personas apliquen la misma operación al mismo tiempo, por ejemplo asignar las últimas unidades de la bodega a dos salones a la vez.

2.9 Un script que se puede repetir

Cada tabla se crea con `IF OBJECT_ID(...) IS NULL CREATE TABLE`, y las vistas, funciones, procedimientos y triggers con `CREATE OR ALTER`. Por eso puedes ejecutar el script otra vez sin destruir nada. Esa propiedad se llama idempotencia, y es una buena respuesta cuando te pregunten cómo se actualiza la base.

3. Las tablas: cómo se crean y qué las protege

Las tablas son el cimiento: si están bien diseñadas, los datos incorrectos ni siquiera pueden entrar. Aquí aprendes a leer un `CREATE TABLE` y recorres las tablas que más te van a preguntar.

3.1 Cómo leer un CREATE TABLE

Todas siguen el mismo patrón. Primero el nombre, luego las columnas con su tipo, y al final las restricciones con nombre propio (`PK_` para llave primaria, `FK_` para foránea, `CK_` para chequeo, `UQ_` para único). Ponerles nombre sirve para que el mensaje de error diga cuál regla falló.

```
IF OBJECT_ID(N'dbo.Prestamos', 'U') IS NULL
CREATE TABLE dbo.Prestamos (
    IdPrestamo   INT IDENTITY(1,1) NOT NULL,
    IdUnidad     INT NULL,
    Estado       VARCHAR(10) NOT NULL CONSTRAINT DF_Prestamos_Estado DEFAULT
'Activo',
    FechaLimite  DATETIME NOT NULL,
    CONSTRAINT PK_Prestamos PRIMARY KEY (IdPrestamo),
    CONSTRAINT FK_Prestamos_Unidad FOREIGN KEY (IdUnidad) REFERENCES
dbo.UnidadesActivo(IdUnidad),
    CONSTRAINT CK_Prestamos_Estado CHECK ([Estado]='Devuelto' OR
[Estado]='Activo')
);
```

Este fragmento resume `Prestamos` (el real tiene más columnas). Léelo así: cada préstamo tiene un número propio; apunta a una unidad que debe existir; nace "Activo"; y el estado solo puede ser "Activo" o "Devuelto".

3.2 Empleados y Usuarios: personas y cuentas

Son dos cosas distintas. `Empleados` es el personal de la institución; `Usuarios` son las cuentas que pueden entrar al sistema. Un conserje es empleado aunque nunca tenga usuario.

En `Empleados` hay tres protecciones que debes saber explicar:

- `UNIQUE (Nombre)`: no puede haber dos empleados con el mismo nombre.
- `CHECK` del DUI: el formato debe ser ocho dígitos, un guión y un dígito. Un índice único filtrado asegura que dos empleados no compartan DUI, pero deja pasar los que no lo tienen (el DUI es opcional).
- Llaves foráneas a `Cargos`, `EstadosEmpleado` y `Planillas`. El estado 2 significa Inactivo y es lo que usa la baja lógica.

En `Usuarios`, la columna `Clave` guarda un hash BCrypt, nunca la contraseña. Un hash es una huella de un solo sentido: sirve para comprobar si una contraseña coincide, pero no se puede revertir. También están `Activo` (para desactivar sin borrar) y `DebeCambiarClave` (para forzar una clave nueva tras un restablecimiento).

Empleados es el centro del personal, los pagos y la carga



relaciones de Empleados · 7 tablas, 6 llaves foráneas

Las tablas que apuntan a `Empleados` (pagos, materias y carga) no pueden tener una fila sin un empleado real. `Cargos` y `Planillas` son catálogos a los que el empleado hace referencia. `AsignacionesDocente` también apunta a `Materias` y a `Secciones`, y `EmpleadoFotos` también cuelga de `Empleados`; se dejaron fuera para que el dibujo se lea de un vistazo.

3.3 Salones y la bodega

`Salones` tiene una peculiaridad: `IdSalon` no es `IDENTITY`; el procedimiento `sp_GuardarSalon` calcula el siguiente número como el máximo más uno. Tiene `CHECK` para que el tipo sea uno de siete valores permitidos (Aula, Laboratorio, Bodega...) y para que la capacidad esté entre 1 y 500.

La **bodega** no es una tabla aparte: es un salón con `Tipo = 'Bodega'`. La función `fn_IdBodega()` la encuentra. Todo el equipo nuevo entra ahí y después se asigna a los salones.

3.4 Inventario: catálogo y unidades

Esta es la parte más valiosa del diseño y la que más conviene explicar bien.

- `GestionActivos` es el **catálogo**: "Pupitre", "Proyector", "Laptop". Una fila por tipo de equipo.
- `UnidadesActivo` son las **piezas físicas**: este pupitre, con su código `ACT-0004-001`, su estado, su salón, su costo, su proveedor y, si aplica, su fecha y motivo de baja.

La relación es de uno a muchos con `ON DELETE CASCADE`. El código de inventario tiene un índice único filtrado: no se repite, pero puede estar vacío. Y `CostoUnitario` lleva un `CHECK` para que nunca sea negativo.

Cada pieza física cuelga de un tipo de equipo y vive en un salón



relaciones de UnidadesActivo · 7 tablas, 7 llaves foráneas

Una unidad pertenece a un tipo de equipo (`GestionActivos`) y está en un salón, y por eso el estado, la ubicación y el historial son de cada pieza y no del tipo. Los préstamos, los reportes de daño y los mantenimientos cuelgan de la unidad. `Salones` y `Prestamos` también apuntan a `Empleados`, que es donde el inventario y el personal se encuentran. Para que se lea de un vistazo se omitieron el catálogo de estados y otras llaves, como la que une `Prestamos` con `GestionActivos`.

Al lado hay dos tablas de historial: `AuditoriaUnidades` y `AuditoriaActivos`. Las llenan triggers (sección 6).

3.5 Préstamos, reportes de daño y mantenimientos

Las tres cuelgan de `UnidadesActivo`. Lo que debes retener son sus reglas:

Tabla	Regla que la protege	Cómo se aplica
`Prestamos`	Una unidad no puede tener dos préstamos activos	Índice único filtrado `WHERE Estado = 'Activo'`

Tabla	Regla que la protege	Cómo se aplica
`Prestamos`	El tipo de responsable es Docente, Personal administrativo o Alumno	`CHECK` en `TipoResponsable`
`ReportesDanio`	Una unidad no puede tener dos reportes abiertos	Índice único filtrado `WHERE Estado = 'Abierto'`
`ReportesDanio`	La descripción no puede ser solo espacios	`CHECK` con `LTRIM` y `RTRIM`
`MantenimientosActivo`	El tipo es Preventivo o Correctivo y el costo no es negativo	Dos `CHECK`

En `Prestamos`, si el responsable es Docente o Personal administrativo, `IdEmpleado` apunta a un empleado real. Si es Alumno, se guarda solo el nombre como texto. Esa regla la aplica `sp_RegistrarPrestamo`.

3.6 Consumibles y su kardex

`Consumibles` guarda cuánto hay (`StockActual`) y el mínimo recomendado (`StockMinimo`), ambos con `CHECK` para que no sean negativos. `MovimientosConsumible` es el **kardex**: cada entrada o salida es una fila con la cantidad, el motivo, el destino y el saldo resultante. Se guarda el saldo en cada movimiento para que el historial se pueda leer sin recalcular.

3.7 Planillas, pagos y cierre de mes

Hay una distinción clave que te conviene tener clara. `Planillas` es el **catálogo de salarios** (estructuras con base, bono y descuento) y cada empleado tiene asignada una. `PagosPlanilla` es lo que realmente se paga cada mes.

En `PagosPlanilla` hay cuatro protecciones: el período siempre es el día 1 del mes (`CHECK`), los montos son coherentes (`CHECK`), `SalarioNeto` es columna calculada y existe un único pago vigente por empleado y mes. `PeriodosPlanilla` guarda qué meses están cerrados, y `PeriodosPlanillaHistorial` anota quién cerró o reabrió cada mes.

`Ausencias` alimenta el descuento: tiene tipo (Falta, Permiso, Incapacidad), si está justificada, si es con goce de sueldo y un `CHECK` para que la fecha final no sea anterior a la inicial. Se anula con una bandera `Anulada` en lugar de borrarla.

3.8 Auditoria: el registro que no se borra

`Auditoria` registra quién, cuándo, en qué tabla y qué cambió. Guarda el contenido antes y después como texto JSON. Tiene un `CHECK` que limita la operación a cuatro letras (I insertar, U modificar, D borrar, A acceso) y varios índices para buscar rápido por fecha, usuario o tabla.

3.9 Datos de ejemplo

Además de las tablas, el script carga datos iniciales (roles, estados, cargos, grados) y datos de ejemplo: 50 empleados, 8 materias, 5 estructuras salariales, 13 salones más la bodega, 31 tipos de equipo y 692 unidades. Los de ejemplo se insertan solo si faltan. El script no crea usuarios: el primer administrador se crea desde el asistente de primer uso del programa.

4. Funciones y vistas: consultar sin repetir trabajo

Las funciones calculan y las vistas presentan. Ninguna de las dos guarda datos propios: siempre trabajan sobre las tablas, así que lo que muestran está al día.

4.1 Funciones

Una función recibe valores y devuelve un resultado. Tu script tiene seis y las puedes clasificar en dos tipos.

- **Escalares:** devuelven un solo valor (un número, una fecha, un sí o no). Son cinco: `fn_DiasHabiles`, `fn_IdBodega`, `fn_PeriodoCerrado`, `fn_DiasDescontados` y `fn_DescuentoAusencias`.
- **De tabla:** devuelven un conjunto de filas, como si fuera una tabla. Es `fn_AuditoriaCambios`, que recibe el número de un registro de auditoría y devuelve una fila por cada campo que cambió.

La mejor forma de entenderlas es ver cómo se encadenan para calcular el descuento por ausencias:

1. `fn_DiasHabiles(inicio, fin)` cuenta los días de lunes a viernes entre dos fechas. No usa el idioma del servidor: parte de que el 1 de enero de 1900 fue lunes y calcula por módulo 7.
2. `fn_DiasDescontados(empleado, período)` suma los días hábiles que caen dentro del mes en las faltas injustificadas y los permisos sin goce de sueldo.

3. `fn_DescuentoAusencias(empleado, período, salario)` divide el salario entre los días hábiles del mes y lo multiplica por los días descontados.

Un ejemplo con números: salario base de 660, mes con 22 días hábiles, dos faltas sin justificar. Cada día vale 660 ÷ 22 = 30, así que el descuento es 2 × 30 = 60. Si una ausencia empieza el 29 del mes anterior y termina el 3 del mes actual, solo se cuenta la parte que cae en el mes del pago.

Otras dos funciones son pequeñas pero se usan mucho. `fn_PeriodoCerrado(fecha)` responde si el mes está cerrado, y los procedimientos de pagos y ausencias la consultan antes de permitir cambios. `fn_IdBodega()` devuelve el número del salón bodega.

¿Por qué función y no código repetido? Porque la regla vive en un solo sitio. Si mañana cambia la forma de contar días hábiles, se corrige `fn_DiasHabiles` y todos los procedimientos y vistas que la usan quedan corregidos.

4.2 Vistas

Una vista es una consulta guardada con nombre. Escribir `SELECT * FROM vw_Prestamos` es como ejecutar la consulta completa que está dentro, con sus uniones y cálculos. La vista no guarda copia de nada.

Sirve para tres cosas:

- **Simplificar.** La pantalla de préstamos necesita el nombre del equipo, el aula, el estado de devolución y una alerta. Eso son varias uniones; la vista las hace una vez y la pantalla solo la consulta.
- **Centralizar lógica.** Una regla calculada en la vista se ve igual en todas las pantallas.
- **Ordenar el acceso.** La pantalla lee vistas y escribe mediante procedimientos, así que cada cosa tiene su camino.

Este es el ejemplo más claro de lógica dentro de una vista. La columna `Alerta` de `vw_Prestamos` se calcula con un `CASE` en cada consulta:

```
CASE WHEN p.Estado = 'Devuelto'                       THEN 'Devuelto'
     WHEN p.FechaLimite < GETDATE()                   THEN 'Atrasado'
     WHEN p.FechaLimite < DATEADD(DAY, 2, GETDATE()) THEN 'Por vencer'
     ELSE 'En curso' END AS Alerta
```

Como usa `GETDATE()`, un préstamo pasa de "En curso" a "Por vencer" y luego a "Atrasado" sin que nadie actualice nada. Por eso la alerta no es una columna guardada: se calcula al mirar. De ahí salen los colores de la pantalla de préstamos.

Otro caso parecido es `vw_StockConsumibles`: `Agotado` si el stock es cero, `Stock bajo` si es menor o igual al mínimo y `Normal` en otro caso. La pantalla de consumibles pinta esa columna.

4.3 Las vistas del sistema, por familias

Las 27 vistas se dividen en cuatro familias. No hace falta memorizarlas una por una; basta con saber a cuál pertenece cada una.

Familia	Vistas	Qué hacen
Detalle	`vw_UnidadesDetalle`, `vw_ActivosDetalle`, `vw_EmpleadosDetalle`, `vw_PagosPlanillaDetalle`, `vw_Prestamos`, `vw_Mantenimientos`, `vw_ReportesDanio`, `vw_AusenciasDetalle`	Unen tablas y traen nombres en lugar de números
Resúmenes agrupados	`vw_ResumenActivosPorEstado`, `vw_ResumenActivosPorCategoria`, `vw_ResumenEmpleadosPorEstado`, `vw_ResumenPlanillaPorCargo`, `vw_ResumenActivosPorSalon`, `vw_ResumenEstadoPorSalon`	Agrupan y cuentan por estado, categoría, salón o cargo. Hoy ninguna pantalla las llama (el inicio hace sus propias consultas): sirven para consultas y reportes directos
Disponibilidad y stock	`vw_StockBodega`, `vw_StockConsumibles`, `vw_KardexConsumibles`, `vw_CargaDocente`	Responden "cuánto hay" o "cuánto queda"
Auditoría	`vw_Auditoria`, `vw_AuditoriaUnidadesDetalle`, `vw_AuditoriaActivosDetalle`, `vw_HistorialMovimientos`	Convierten el historial técnico en frases legibles; vw_HistorialMovimientos junta préstamos, devoluciones, consumibles y mantenimientos en una sola lista

Hay vistas que se apoyan en otras: `vw_CargaDocente` agrupa sobre `vw_AsignacionesDetalle`. También `vw_StockBodega` usa `fn_IdBodega()` para saber qué salón es la bodega y descarta las unidades en mal estado o ya prestadas, así que "Disponibles" es exactamente lo asignable.

4.4 Una vista que vale la pena entender: vw_Auditoria

Es la vista que alimenta la pantalla de Registro de actividad. Traduce el nombre técnico de la tabla a un módulo en español (por ejemplo `PagosPlanilla` se muestra como "Pagos de planilla"), decide la acción en lenguaje humano ("Creó", "Modificó", "Eliminó", "Inició sesión") y arma el resumen con `fn_AuditoriaCambios`. Es un buen ejemplo de cómo la base de datos entrega el dato ya listo para mostrar.

5. Procedimientos almacenados: las reglas de negocio

Un procedimiento almacenado es un programa guardado dentro de la base de datos que ejecuta una operación completa. Es la pieza donde viven las reglas del negocio, y es la que más te van a preguntar.

5.1 Por qué existen

La pantalla podría ejecutar un `INSERT` directo, pero entonces cada pantalla tendría que acordarse de todas las reglas. Con un procedimiento, la regla se escribe una vez y todas las pantallas que lo llaman la cumplen. Además, el procedimiento corre dentro del servidor, así que valida y escribe en una sola ida y vuelta, y se puede proteger con una transacción.

5.2 Anatomía de uno: sp_RegistrarMovimientoConsumible

Es corto y tiene todas las partes. Léelo en este orden:

```
CREATE OR ALTER PROCEDURE dbo.sp_RegistrarMovimientoConsumible
    @IdConsumible INT, @Tipo VARCHAR(10), @Cantidad INT, @Motivo VARCHAR(40),
    @Destino VARCHAR(100) = NULL, @Usuario VARCHAR(50), @IdSalon INT = NULL
AS
BEGIN
    SET NOCOUNT ON; SET XACT_ABORT ON;
    BEGIN TRY
        IF @Tipo NOT IN ('Entrada', 'Salida')
            THROW 50040, 'El tipo de movimiento debe ser Entrada o Salida.',
1;
        IF @Cantidad IS NULL OR @Cantidad <= 0
            THROW 50041, 'La cantidad debe ser mayor que cero.', 1;

        BEGIN TRANSACTION;
        DECLARE @Stock INT;
```

```
        SELECT @Stock = StockActual FROM dbo.Consumibles WITH (UPDLOCK,
HOLDLOCK)
        WHERE IdConsumible = @IdConsumible;
        IF @Tipo = 'Salida' AND @Cantidad > @Stock
            THROW 50043, 'Stock insuficiente: no se puede despachar más de lo
que hay.', 1;

        DECLARE @Nuevo INT = CASE WHEN @Tipo = 'Entrada' THEN @Stock +
@Cantidad ELSE @Stock - @Cantidad END;
        UPDATE dbo.Consumibles SET StockActual = @Nuevo WHERE IdConsumible =
@IdConsumible;
        INSERT INTO dbo.MovimientosConsumible (IdConsumible, Tipo, Cantidad,
Motivo, Destino, SaldoResultante, Usuario, IdSalon)
        VALUES (@IdConsumible, @Tipo, @Cantidad, @Motivo, @Destino, @Nuevo,
@Usuario, @IdSalon);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

El fragmento omite alguna validación del original, pero conserva la estructura. Estas son las partes:

1. **Parámetros (`@Algo`).** Lo que la pantalla le envía. Los que tienen `= NULL` son opcionales.
2. **`SET NOCOUNT ON`.** Evita mensajes de "N filas afectadas" que no sirven al programa.
3. **`SET XACT_ABORT ON`.** Si ocurre un error, cancela la transacción completa.
4. **Validaciones primero.** Cada regla incumplida lanza un `THROW` con un código propio y un mensaje en español. Todo lo anterior al `BEGIN TRANSACTION` no ha tocado datos.
5. **Transacción con bloqueo.** `UPDLOCK, HOLDLOCK` retiene la fila de stock mientras se calcula. Sin eso, dos salidas simultáneas leerían el mismo stock y las dos pasarían.
6. **Escritura doble.** Actualiza el stock y agrega el movimiento al kardex. Ambos cambios se guardan juntos o ninguno.
7. **`CATCH`.** Si algo falla, deshace y vuelve a lanzar el error para que C# lo reciba.

El `CHECK (StockActual >= 0)` de la tabla es una segunda barrera: aunque el procedimiento tuviera un error, el stock nunca quedaría negativo.

5.3 THROW y los códigos de error

`THROW` detiene la ejecución y envía un error al programa. Los números 50000 en adelante son para errores propios. Tu proyecto los agrupa por tema, y eso te ayuda a ubicarlos:

Rango	Tema	Ejemplo
50031 a 50037	Préstamos y devoluciones	50035: la unidad ya está prestada
50040 a 50043	Consumibles	50043: stock insuficiente
50200 a 50207	Carga académica	50204: supera 40 horas
50300 a 50311	Pagos	50306: ya existe un pago en ese período
50500 a 50514	Alta y asignación de unidades	50514: no hay tantas unidades disponibles
50800 a 50808	Cierre de mes	50800: el mes está cerrado
50910 a 50914, 50962	Cambios de estado y bajas	50914: una baja no se revierte
50950 a 50953	Responsable de un préstamo	50952: el empleado no es docente

En el programa, el mensaje del `THROW` es el texto que ve el usuario. Por eso están escritos en español y pensados para el usuario final. El script tiene un anexo con la lista completa.

5.4 Los procedimientos que debes saber explicar

sp_RegistrarPrestamo. Primero decide a quién se presta. Si el responsable es Docente o Personal administrativo exige un empleado real, activo, y comprueba que su cargo corresponde con el tipo (un docente no puede registrarse como administrativo). Si es Alumno, acepta texto libre. Después valida la fecha límite, que la unidad exista, que no esté en mal estado, en reparación ni de baja, y que no tenga otro préstamo activo. Solo entonces inserta. El nombre del responsable lo toma del registro del empleado, no de lo que escribió el usuario.

sp_RegistrarDevolucion. Dentro de una transacción cierra el préstamo y cambia el estado de esa unidad. Si la unidad vuelve como "Dado de baja" exige un motivo de al menos cinco caracteres y guarda fecha y motivo. Una unidad ya dada de baja no puede volver a otro estado.

sp_CambiarEstadoUnidades. Recibe una lista de números separados por coma (`@Ids`), la convierte en filas con `STRING_SPLIT` y valida: no se da de baja una unidad prestada, no se marca "Bueno" una unidad con reporte de daño abierto, y una baja no se revierte. La fecha y el motivo de baja se escriben cuando la unidad entra en baja y se conservan si ya lo estaba.

sp_AsignarUnidades. Pasa unidades de la bodega a un salón. Cuenta las disponibles con bloqueo y, si piden más de las que hay, rechaza con el 50514. Mueve exactamente las unidades pedidas y devuelve cuántas asignó gracias a `OUTPUT`.

sp_RegistrarPago. Crea o edita un pago. Lo primero que revisa es que el mes no esté cerrado (usa `fn_PeriodoCerrado`). Luego valida montos, que el período no sea futuro, que no haya otro pago vigente del mismo empleado en el mes y que un pago ya pagado no se edite.

sp_GenerarPlanilla. Crea los pagos pendientes de todos los empleados activos para un mes. Tiene un modo simulación (`@Simular = 1`) que solo cuenta cuántos generaría, cuántos ya existen y cuántos empleados no tienen salario; la pantalla lo usa como vista previa. Para cada empleado suma el descuento de ley de su estructura más el descuento por ausencias, limitado para que nunca supere base más bono.

sp_CerrarPeriodo. Cierra un mes solo si tiene pagos y no queda ninguno pendiente. Registra el cierre en `PeriodosPlanilla` y en el historial, todo en una transacción. `sp_ReabrirPeriodo` hace lo inverso.

sp_AsignarDocente. Valida que el empleado sea docente, que la materia sea una de las suyas y que no se repita; con bloqueo comprueba que el total de horas del docente en el año no pase de 40. El tope cuenta todas sus materias, no solo la de este grupo.

5.5 Los 34 procedimientos, por familias

Familia	Procedimientos
Usuarios y acceso (8)	`sp_CrearUsuario`, `sp_ActualizarUsuario`, `sp_CambiarEstadoUsuario`, `sp_CambiarMiClave`, `sp_RestablecerClaveUsuario`, `sp_RegistrarAcceso`, `sp_RegistrarEvento`, `sp_GuardarConfiguracionInicial`
Personal y académico (3)	`sp_GuardarMateria`, `sp_GuardarMateriasEmpleado`, `sp_AsignarDocente`

Familia	Procedimientos
Planillas y ausencias (9)	`sp_RegistrarPago`, `sp_MarcarPagado`, `sp_AnularPago`, `sp_GenerarPlanilla`, `sp_CerrarPeriodo`, `sp_ReabrirPeriodo`, `sp_RegistrarAusencia`, `sp_AnularAusencia`, `sp_RecalcularDescuentoAusencias`
Activos (10)	`sp_AltaUnidades`, `sp_AgregarUnidades`, `sp_AsignarUnidades`, `sp_MoverUnidades`, `sp_QuitarUnidades`, `sp_CambiarEstadoUnidades`, `sp_CambiarEstadoUnidad`, `sp_RegistrarPrestamo`, `sp_RegistrarDevolucion`, `sp_RegistrarMantenimiento`
Salones (2)	`sp_GuardarSalon`, `sp_CambiarEstadoSalon`
Consumibles (1)	`sp_RegistrarMovimientoConsumible`
Auditoría (1)	`sp_CrearTriggerAuditoria`

6. Triggers y auditoría: la base que se vigila sola

Un trigger es código que la base ejecuta automáticamente cuando alguien inserta, modifica o borra filas de una tabla. Nadie lo llama: se dispara solo. Eso lo hace perfecto para el historial, porque nadie puede olvidarse de registrar un cambio.

6.1 Las dos tablas mágicas: inserted y deleted

Dentro de un trigger existen dos tablas temporales que el motor llena por ti.

Operación	`inserted` contiene	`deleted` contiene
`INSERT`	Las filas nuevas	Nada
`UPDATE`	Las filas con los valores nuevos	Las mismas filas con los valores viejos
`DELETE`	Nada	Las filas borradas

Con eso el trigger sabe qué pasó. Si hay filas en las dos, fue una modificación; solo en `inserted`, un alta; solo en `deleted`, una baja. Además, un solo `UPDATE` puede cambiar muchas filas, y el trigger se dispara una sola vez con todas. Por eso se escribe con consultas sobre conjuntos y no fila por fila.

Tus triggers son `AFTER`: se ejecutan después de que el cambio se aplicó, dentro de la misma transacción. Si el trigger falla, el cambio completo se deshace.

6.2 Quién hizo el cambio: SESSION_CONTEXT

El trigger necesita saber qué usuario del sistema actuó. El login de Windows no sirve, porque todos los usuarios del programa se conectan con la misma cuenta del equipo. La solución son dos piezas que trabajan juntas.

1. En C#, `Conexion.Conectar()` suscribe un evento a la conexión. Cada vez que se abre, ejecuta `sp_set_session_context @key = 'Usuario', @value = <usuario de la sesión>`.
2. En SQL, el trigger lee ese valor con `SESSION_CONTEXT(N'Usuario')` y lo guarda como autor del cambio.

El valor vive mientras dure esa conexión, y no lo comparte con otras. Así, el registro dice "mgarcia modificó el pago" y no "el equipo de la oficina".

6.3 El trigger de auditoría, paso a paso

Hay 17 triggers con este mismo molde, uno por tabla auditada (`trg_Aud_Empleados`, `trg_Aud_Prestamos`...). Este es el esqueleto:

```
CREATE OR ALTER TRIGGER dbo.trg_Aud_Empleados ON dbo.Empleados AFTER INSERT,
UPDATE, DELETE AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @op CHAR(1) = CASE WHEN EXISTS (SELECT 1 FROM inserted) AND
EXISTS (SELECT 1 FROM deleted) THEN 'U'
                               WHEN EXISTS (SELECT 1 FROM inserted) THEN 'I'
ELSE 'D' END;

    INSERT INTO dbo.Auditoria (Usuario, Tabla, Operacion, Clave, Antes,
Despues)
    SELECT CAST(SESSION_CONTEXT(N'Usuario') AS NVARCHAR(50)), N'Empleados',
@op,
           CAST(COALESCE(i.IdEmpleado, d.IdEmpleado) AS NVARCHAR(100)),
           x.Antes, x.Despues
    FROM inserted i FULL OUTER JOIN deleted d ON d.IdEmpleado = i.IdEmpleado
    CROSS APPLY (SELECT
        Antes   = CASE WHEN @op = 'I' THEN NULL ELSE (SELECT d.Nombre,
d.IdCargo /* ... */ FOR JSON PATH, WITHOUT_ARRAY_WRAPPER) END,
        Despues = CASE WHEN @op = 'D' THEN NULL ELSE (SELECT i.Nombre,
i.IdCargo /* ... */ FOR JSON PATH, WITHOUT_ARRAY_WRAPPER) END) x
    WHERE @op <> 'U' OR ISNULL(x.Antes, N'') <> ISNULL(x.Despues, N'');
END
```

En el original aparecen todas las columnas de `Empleados`; aquí se abrevió. Las piezas importantes:

- **`@op`.** Decide si fue inserción (`I`), modificación (`U`) o borrado (`D`) mirando qué tablas mágicas tienen filas.
- **`FULL OUTER JOIN`.** Une `inserted` y `deleted` por la llave. Así una misma consulta sirve para los tres casos: en un alta falta el lado `deleted`, en una baja falta `inserted`.
- **`FOR JSON PATH`.** Convierte la fila en un texto JSON, por ejemplo `{"Nombre":"Ana","IdCargo":3}`. Se guarda un "antes" y un "después" completos.
- **El `WHERE` final.** Si fue una modificación pero el antes y el después son iguales, no se registra nada. Evita ruido cuando un `UPDATE` no cambia valores reales.

Se guarda la fila entera y no solo la diferencia porque es más simple y más fiable. La pantalla luego muestra solo lo que cambió gracias a `fn_AuditoriaCambios`.

6.4 El generador: sp_CrearTriggerAuditoria

Escribir 17 triggers a mano sería repetitivo y propenso a errores. Por eso existe un procedimiento que los fabrica. Recibe el nombre de una tabla, busca sus columnas y su llave primaria en el catálogo del sistema, arma el texto del trigger como cadena y lo ejecuta con `sp_executesql`. Esa técnica se llama SQL dinámico.

Tiene dos detalles de diseño que conviene mencionar. Omite las columnas binarias (fotos y logotipo), que no tendrían sentido en un historial. Y acepta una lista de columnas a excluir: en `Usuarios` se excluyen `Clave` y `UltimoAcceso`, así que ni siquiera el hash de las contraseñas se copia al registro.

6.5 Un registro que no se puede alterar

Un historial que cualquiera puede editar no sirve como evidencia. `trg_Auditoria_Inmutable` es un trigger `INSTEAD OF UPDATE, DELETE` sobre `Auditoria`: en lugar de ejecutar el cambio, lanza el error 50900. Nadie puede modificar ni borrar el registro desde el sistema.

La palabra clave es `INSTEAD OF` (en lugar de). A diferencia de `AFTER`, el trigger reemplaza a la operación original. Aquí el reemplazo es simplemente rechazarla.

6.6 Historial del inventario

`trg_AuditoriaUnidades` y `trg_AuditoriaActivos` son más especializados: guardan en columnas normales (no JSON) el estado y el salón anterior y nuevo de cada unidad. Sirven para responder "¿dónde estuvo este pupitre y en qué estado?". Usan `ISNULL(SESSION_CONTEXT(N'Usuario'), SUSER_SNAME())`: si no hay usuario del sistema, caen al login de la conexión.

6.7 Cómo se ve en el programa

La pantalla **Registro de actividad** lee `vw_Auditoria` y muestra fecha, usuario, módulo, acción y un resumen de qué cambió. Los inicios de sesión, intentos fallidos y eventos de usuarios llegan por otro camino: el programa llama a `sp_RegistrarAcceso` y `sp_RegistrarEvento`, que escriben en la misma tabla con la operación `A`. Así, todo queda en un solo lugar.

Una observación honesta para tu defensa: si alguien modifica datos directamente desde SSMS, no hay usuario del sistema en la sesión. El registro sale con usuario vacío y la vista lo muestra como "(sistema)". El cambio igual queda registrado; solo falta saber quién.

7. Cómo se refleja en el sistema (C#)

La base de datos y el programa se hablan a través de una capa muy delgada. Entender esa capa te permite explicar qué pasa cuando alguien pulsa un botón.

7.1 Las capas

El proyecto está dividido en dos. `Vistass` contiene las pantallas (formularios). `Modelos` contiene las clases que no son pantallas: `Conexion`, `Datos`, `Sesion`, `ServicioUsuarios`, los exportadores y otras. Las pantallas llaman a `Modelos`, y `Modelos` habla con SQL Server. Las pantallas más nuevas piden a `Datos` que consulte; algunas más antiguas abren la conexión con `Conexion.Conectar()` directamente. Las dos rutas usan la misma conexión, así que el usuario de la sesión se registra igual.

7.2 Conexion: abrir la puerta y decir quién eres

`Conexion.Conectar()` arma la cadena de conexión a LocalDB con seguridad integrada, es decir, sin contraseña de base de datos: usa la cuenta de Windows. Además suscribe un evento que se dispara cada vez que la conexión pasa de cerrada a abierta. Ese evento ejecuta `sp_set_session_context` con el usuario que inició sesión en el programa. Es la mitad C# de lo que viste en la sección de triggers.

7.3 Datos: cuatro métodos para todo

La clase `Datos` tiene los métodos que usa casi todo el programa:

Método	Para qué	Ejemplo de uso
`Tabla(sql, parámetros)`	Ejecuta una consulta y devuelve una tabla en memoria (`DataTable`)	Cargar la lista de préstamos desde `vw_Prestamos`
`Escalar(sql, parámetros)`	Devuelve un solo valor	Obtener el stock actual de un consumible
`Ejecutar(sql, parámetros)`	Ejecuta una instrucción sin devolver datos	Un `UPDATE` o `INSERT` sencillo
`Procedimiento(nombre, parámetros)`	Llama a un procedimiento almacenado	`sp_RegistrarPrestamo`

Cada llamada abre su conexión, ejecuta y la cierra (por el `using`). No hay una conexión eterna abierta.

7.4 Parámetros: por qué nunca se pega texto al SQL

Observa cómo se llama a un procedimiento:

```
Datos.Procedimiento("sp_RegistrarMovimientoConsumible",
    Datos.P("@IdConsumible", id),
    Datos.P("@Tipo", cmbMovTipo.Text),
    Datos.P("@Cantidad", (int)numMovCant.Value),
    Datos.P("@Motivo", cmbMovMotivo.Text),
    Datos.PTexto("@Destino", destino),
    Datos.P("@Usuario", UsuarioActual()),
    Datos.P("@IdSalon", idSalon));
```

`Datos.P` crea un parámetro. Los valores viajan separados del texto de la instrucción, y el motor nunca los interpreta como código. Eso previene la **inyección SQL**: si alguien escribe `'; DROP TABLE Empleados; --` en un cuadro de texto, se guarda como texto, no se ejecuta. `Datos.PTexto` además convierte un texto vacío en `NULL`.

7.5 De la base a la tabla de la pantalla

Cuando abres Gestión de personal ocurre esta cadena:

1. La pantalla llama a `Datos.Tabla` con un `SELECT` sobre las tablas o una vista.
2. SQL Server devuelve las filas y se guardan en un `DataTable` en memoria.
3. Se envuelve en un `DataView`, que permite filtrar y ordenar sin volver a la base. Por eso el buscador de las pantallas responde al instante: modifica el `RowFilter` de la vista de datos.
4. La tabla visual (`GridOscuro`) muestra el `DataView`.

Las columnas de color vienen de la base. La alerta de préstamos y de consumibles se calcula en SQL (las vistas); la pantalla solo traduce la palabra "Atrasado" o "Agotado" a un color.

7.6 Qué pasa cuando algo falla

Si un procedimiento lanza un `THROW`, C# recibe una `SqlException`. La pantalla la atrapa y muestra su `Message`, que es el texto en español que escribiste en SQL. Así no hay que traducir códigos: la regla y su mensaje viven juntos en la base.

Para las restricciones de las tablas el mensaje es el de SQL Server, que es técnico. Por eso en algunos puntos el programa reconoce el número de error. Por ejemplo, al crear un consumible, si el número es 2601 o 2627 (valor duplicado en un índice o restricción única), muestra "Ya existe un consumible con ese nombre" junto al campo.

7.7 Qué pantalla usa qué

Este cuadro te ayuda a pasar de lo que ves a lo que ocurre por debajo.

Pantalla	Lee de	Escribe mediante	Se registra en el historial
Préstamos	`vw_Prestamos`	`sp_RegistrarPrestamo`, `sp_RegistrarDevolucion`	`trg_Aud_Prestamos`, `trg_AuditoriaUnidades`
Unidades de un equipo	`vw_UnidadesDetalle`	`sp_CambiarEstadoUnidades`, `sp_AltaUnidades`	`trg_Aud_UnidadesActivo`
Mantenimiento	`vw_Mantenimientos`	`sp_RegistrarMantenimiento`	`trg_Aud_MantenimientosActivo`
Consumibles	`vw_StockConsumibles`, `vw_KardexConsumibles`	`sp_RegistrarMovimientoConsumible`	El propio kardex
Asignar equipo a un salón	`vw_StockBodega`	`sp_AsignarUnidades`	`trg_AuditoriaUnidades`
Pagos de planilla	`vw_PagosPlanillaDetalle`	`sp_RegistrarPago`, `sp_MarcarPagado`, `sp_AnularPago`	`trg_Aud_PagosPlanilla`
Generar planilla y cerrar mes	La propia consulta de simulación	`sp_GenerarPlanilla`, `sp_CerrarPeriodo`, `sp_ReabrirPeriodo`	`trg_Aud_PeriodosPlanilla`
Ausencias	`vw_AusenciasDetalle`	`sp_RegistrarAusencia`, `sp_AnularAusencia`	`trg_Aud_Ausencias`
Carga académica	`vw_CargaDocente`	`sp_AsignarDocente`	`trg_Aud_AsignacionesDocente`
Registro de actividad	`vw_Auditoria`	Nadie: solo lectura	Es el historial mismo
Inicio de sesión	`Usuarios`	`sp_RegistrarAcceso`	`Auditoria` con operación A

7.8 Lo que todavía no pasa por procedimientos

Ser honesto con esto te favorece en la defensa. Las operaciones más sencillas, como crear o editar un cargo, crear un consumible, o dar de baja lógica a un empleado (pasarlo al estado 2), se hacen con instrucciones `INSERT` o `UPDATE` directas desde las pantallas. Siguen protegidas por las restricciones de la tabla (por ejemplo, el nombre único) y por los triggers, así que el historial se guarda igual. Lo que no tienen es una regla de negocio compleja que valide por encima. Las operaciones con reglas serias, como préstamos, bajas, pagos y planillas, sí pasan siempre por procedimientos.

8. Tres flujos de punta a punta

Cuando domines estos tres recorridos podrás explicar cualquier otra parte del sistema, porque todas siguen el mismo patrón: la pantalla pide, el procedimiento valida y escribe, las restricciones y los triggers vigilan, y la vista devuelve el resultado.

8.1 Prestar un equipo y devolverlo

El préstamo

1. En la pestaña Préstamos eliges el equipo. La pantalla carga en una lista las unidades de ese equipo que se pueden prestar: se excluyen las de estado malo, en reparación o dadas de baja, y las que ya tienen un préstamo activo.
2. Eliges el tipo de responsable. Si es Docente o Personal administrativo, la lista pasa a ser de empleados reales; si es Alumno, escribes el nombre.
3. Al pulsar el botón, C# hace una primera revisión rápida (campos vacíos, fecha límite futura) y muestra los avisos junto a cada campo.
4. Si pasa, llama a `sp_RegistrarPrestamo` con la unidad, el tipo, el empleado, el aula, la fecha límite y el usuario que entrega.
5. El procedimiento repite las validaciones importantes en el servidor (empleado activo, cargo coherente, unidad en buen estado, sin préstamo activo) e inserta en `Prestamos`.
6. El índice único `UX_Prestamos_UnidadActiva` es la última red: si dos personas pidieran la misma unidad en el mismo instante, la segunda inserción fallaría.
7. El trigger `trg_Aud_Prestamos` anota quién creó el préstamo y con qué datos.
8. La pantalla recarga `vw_Prestamos`. La columna `Alerta` se calcula en ese momento: "En curso", "Por vencer" o "Atrasado".

La devolución

1. Seleccionas el préstamo, eliges el estado físico con que vuelve el equipo y pulsas Registrar devolución.
2. `sp_RegistrarDevolucion` abre una transacción y bloquea la fila del préstamo.
3. Verifica que siga activo. Si el estado nuevo es "Dado de baja", exige un motivo de al menos cinco caracteres.
4. Cierra el préstamo (`Estado = 'Devuelto'`, fecha de devolución) y actualiza `UnidadesActivo` con el estado nuevo. Si entra en baja, guarda la fecha y el motivo.
5. Hace `COMMIT`. El trigger de unidades anota el cambio de estado en `AuditoriaUnidades`.

Qué puede salir mal y cómo reacciona

- Prestar una unidad que ya está prestada: error 50035, que la pantalla muestra tal cual.
- Devolver una unidad dada de baja con otro estado: error 50914.
- Devolver como "Dado de baja" sin explicar el motivo: error 50962.

8.2 Una salida de consumible

1. En la pestaña Consumibles eliges el consumible y el tipo (Entrada o Salida). Si es salida, la pantalla consulta el stock actual y limita el numérico de cantidad a ese tope.
2. Según el motivo, el campo de destino cambia: una lista de salones si es "Despacho a aula", texto obligatorio en los demás casos.
3. C# llama a `sp_RegistrarMovimientoConsumible`. Si el destino es un salón, envía también su número, para que el movimiento quede enlazado a ese salón.
4. El procedimiento abre una transacción y bloquea la fila del consumible. Si la salida pide más de lo que hay, lanza el error 50043.
5. Calcula el saldo nuevo, actualiza `Consumibles.StockActual` y agrega una fila al kardex (`MovimientosConsumible`) con el saldo resultante.
6. Hace `COMMIT`. El `CHECK (StockActual >= 0)` queda como segunda barrera.
7. La pantalla recarga `vw_StockConsumibles`. Si el stock quedó en cero o bajo el mínimo, la columna `Alerta` lo marca y el sistema muestra el aviso.

La lección de este flujo es la defensa en capas: el tope del numérico (comodidad), el procedimiento (regla) y el `CHECK` (red final) protegen lo mismo en tres niveles.

8.3 El mes de planilla: de ausencias a cierre

1. **Se registra una ausencia.** `sp_RegistrarAusencia` valida (empleado activo, rango razonable, que incluya días hábiles, que el mes no esté cerrado, que no se cruce con otra) e inserta. Dentro de la misma transacción llama a `sp_RecalcularDescuentoAusencias`, que actualiza el descuento de los pagos pendientes de esos meses.
2. **Se genera la planilla.** La pantalla llama primero a `sp_GenerarPlanilla` con `@Simular = 1` y muestra cuántos pagos se crearían y cuántos empleados no tienen salario asignado. Al confirmar, la llama con `@Simular = 0`. El procedimiento crea un pago pendiente por empleado activo con el salario de su estructura y el descuento por ausencias calculado con las funciones de la sección 4.

3. **Se revisan y se pagan.** Un pago pendiente se puede editar con `sp_RegistrarPago`. `sp_MarcarPagado` lo pasa a pagado y guarda la fecha. Un pago pagado ya no se edita: para corregirlo se anula (`sp_AnularPago`) y se registra de nuevo; el índice filtrado permite eso porque ignora los anulados.
4. **Se cierra el mes.** `sp_CerrarPeriodo` exige que el mes tenga pagos y que ninguno siga pendiente. Escribe en `PeriodosPlanilla` y en el historial.
5. **Después del cierre.** Cualquier intento de crear o modificar pagos o ausencias de ese mes choca con `fn_PeriodoCerrado` y recibe el error 50800 u 50801. Solo un administrador puede reabrir el mes con `sp_ReabrirPeriodo`, y eso también queda en el historial.

Este flujo muestra bien por qué la lógica está en la base: ocho procedimientos, cuatro funciones y cuatro tablas cooperan, y la pantalla no necesita conocer ninguna de esas reglas.

9. Plan de repaso y cómo explicarlo en voz alta

Saber algo y poder explicarlo son dos habilidades distintas. Esta sección te da un plan de tres días, un método para responder cualquier pregunta y una demostración en vivo que puedes hacer en SQL Server.

9.1 Plan de tres días

Día	Lee	Practica
1	Secciones 1, 2 y 3 y las preguntas de los bloques 1 y 2	Dibuja de memoria las relaciones de `Empleados`, `Cargos` y `Planillas`, y las de `GestionActivos`, `UnidadesActivo` y `Prestamos`
2	Secciones 4, 5 y 6 y las preguntas de los bloques 3, 4 y 5	Explica en voz alta qué hace `sp_RegistrarMovimientoConsumible` línea por línea, sin mirar
3	Secciones 7 y 8 y el bloque 6	Narra los tres flujos completos, de la pantalla a la tabla y de vuelta, como si se lo explicaras a alguien que no sabe

9.2 El método de las cuatro preguntas

Para cualquier objeto de la base (una tabla, una vista, un procedimiento, un trigger) responde siempre en este orden. Funciona porque te obliga a conectar el detalle técnico con el sistema.

1. **¿Qué problema resuelve?** Una frase, sin términos técnicos.
2. **¿Cómo lo hace?** El mecanismo, con un ejemplo real de tu script.
3. **¿Dónde se ve en el programa?** La pantalla o el botón que lo usa.
4. **¿Qué pasa si falla o si alguien intenta saltarlo?** La barrera que sigue.

Por ejemplo, con `trg_Auditoria_Inmutable`: (1) evita que alguien altere el historial; (2) es un trigger `INSTEAD OF` que lanza el error 50900; (3) por eso la pantalla de Registro de actividad es solo lectura; (4) si alguien intenta borrar desde SSMS, recibe el error y no se borra nada.

9.3 Frases ancla

Son ideas completas que puedes decir con tus palabras. Si las recuerdas, el resto de la explicación sale sola.

- "Las reglas viven en la base de datos, la pantalla solo las pide y las muestra."
- "Cada pieza física es una fila propia; el catálogo solo dice qué tipo de cosa es."
- "Una operación que toca varias tablas va en una transacción: todo o nada."
- "La alerta no se guarda, se calcula al mirar."
- "El historial lo escriben triggers, así que nadie puede olvidarse de registrar un cambio."
- "Hay tres barreras: la pantalla, el procedimiento y la restricción de la tabla."

9.4 Una demostración en vivo para SSMS

Mostrar que la base se defiende sola impresiona más que describirlo. Estas consultas no dañan datos: o solo leen o están pensadas para fallar. Pruébalas antes en tu equipo y ajusta los números a los que existan en tu base.

```
-- 1. Una vista: la alerta se calcula sola
SELECT TOP 5 IdPrestamo, Activo, FechaLimite, Alerta FROM dbo.vw_Prestamos
ORDER BY IdPrestamo DESC;

-- 2. Una funcion: octubre de 2026 tiene 22 dias habiles
SELECT dbo.fn_DiasHabiles('2026-10-01', '2026-10-31') AS DiasHabiles;

-- 3. Una restriccion CHECK: 'Cocina' no es un tipo de salon valido (debe
fallar)
INSERT INTO dbo.Salones (IdSalon, NombreSalon, Tipo) VALUES (9999, 'Prueba',
'Cocina');

-- 4. Un trigger INSTEAD OF: el registro de actividad no se modifica (debe
fallar con 50900)
UPDATE dbo.Auditoria SET Usuario = 'otro' WHERE IdAuditoria = 1;

-- 5. Una regla de negocio en un procedimiento: pedir mas unidades de las que
hay (debe fallar con 50514)
EXEC dbo.sp_AsignarUnidades @IdActivo = 1, @IdSalonDestino = 2, @Cantidad =
100000;

-- 6. El historial legible
SELECT TOP 5 FechaHora, Usuario, Modulo, Accion, Resumen FROM
dbo.vw_Auditoria ORDER BY IdAuditoria DESC;
```

Cada una ilustra una pieza distinta: vista, función, restricción, trigger, procedimiento y auditoría. Dos de ellas deben fallar, y eso es justo lo que quieres enseñar: el mensaje de error es la prueba de que la regla funciona.

9.5 Si no sabes una respuesta

Es mejor ser preciso que parecer seguro. Una forma honesta de salir es explicar lo que sí sabes del tema y decir lo que no: "Esa parte no la tengo clara; lo que sí puedo explicar es cómo funciona en nuestro caso". Los evaluadores valoran que distingas entre lo que entiendes y lo que no.

10. Preguntas y respuestas, bloque 1: conceptos generales y diseño

Las respuestas están escritas para que entiendas el porqué, no para que las recites. Léelas, cierra el documento e intenta explicarlas con tus palabras.

10.1 ¿Qué es una base de datos relacional y por qué la usaron en vez de guardar todo en archivos?

Una base de datos relacional organiza la información en tablas que se relacionan entre sí mediante llaves. Cada tabla trata de un solo tema (empleados, préstamos, salones) y cuando un dato de una tabla necesita hacer referencia a otro, no lo copia: guarda el número que identifica a esa otra fila. Así, el nombre del cargo "Director" existe una sola vez y todos los empleados que lo tienen apuntan a él.

Con archivos de texto o con hojas de cálculo tendríamos tres problemas serios. El primero es la repetición: el mismo dato escrito en muchos lugares, que tarde o temprano se contradice. El segundo es que nada impide guardar datos absurdos, como un pago con salario negativo. El tercero es que dos personas trabajando a la vez pueden pisarse el trabajo. Un motor como SQL Server resuelve los tres: evita la repetición con las relaciones, valida con restricciones y controla la concurrencia con transacciones y bloqueos.

Elegimos SQL Server porque el sistema está hecho en C# con Windows Forms, que se integra de forma natural con él, y porque LocalDB viene incluido con Visual Studio, así que no hace falta instalar un servidor aparte para trabajar o presentar.

10.2 ¿Cómo está organizada su base de datos?

Tiene 30 tablas que se agrupan en siete partes, que coinciden con las secciones del menú del programa. Personal y acceso guarda empleados, cargos, usuarios y roles. Académico guarda grados, secciones, materias y la carga de cada docente. Planillas guarda los salarios, los pagos de cada mes, el cierre de mes y las ausencias. Activos es la parte más grande: catálogo de equipos, unidades físicas, préstamos, reportes de daño y mantenimientos. Salones tiene una sola tabla pero es el punto donde se encuentran los activos y el personal. Consumibles tiene el stock y su kardex. Y Sistema guarda la configuración de la institución y el registro de actividad.

Sobre esas tablas hay 27 vistas para consultar, 34 procedimientos para escribir con reglas, 6 funciones para cálculos y 20 triggers para el historial. Una forma de resumirlo en una frase: las tablas guardan, las vistas muestran, los procedimientos deciden, las funciones calculan y los triggers vigilan.

10.3 ¿Por qué separaron el catálogo de equipos de las unidades físicas?

Porque "Pupitre" y "este pupitre" no son la misma cosa. El catálogo (`GestionActivos`) dice qué tipos de equipo existen. La unidad (`UnidadesActivo`) es una pieza concreta con su código, su estado, su salón, su costo, su proveedor y su historia.

Al principio el estado vivía en el catálogo. Eso significaba que si un pupitre se rompía, el sistema consideraba que todos los pupitres estaban rotos, o ninguno. Era imposible decir "este está en el aula 3 y está dañado, aquel está en bodega y está nuevo". Al pasar el estado a la unidad, cada pieza tiene el suyo, se puede prestar una a la vez, se puede dar de baja una sin afectar a las demás y el historial dice exactamente qué le pasó a cada una.

Es una decisión de modelado: la información debe vivir en el nivel al que realmente pertenece. Eso es, en esencia, lo que pide la normalización.

10.4 ¿Qué es la integridad de los datos y cómo la garantizan?

Integridad significa que los datos guardados son coherentes y cumplen las reglas del negocio. Se divide en tres tipos. La integridad de entidad dice que cada fila se identifica de forma única: son las llaves primarias. La integridad referencial dice que las referencias apuntan a cosas que existen: son las llaves foráneas, y por ellas no se puede registrar un préstamo de una unidad inexistente. Y la integridad de dominio dice que cada valor es razonable: son los `CHECK`, los tipos de dato y los valores por defecto, como que la capacidad de un salón esté entre 1 y 500.

Todo eso lo aplica el motor siempre, sin importar desde dónde lleguen los datos. Sobre esa base se añaden las reglas que no caben en una restricción, que son las de los procedimientos y los triggers. Así hay capas: lo básico lo garantiza la tabla, lo complejo lo garantiza el procedimiento, y la pantalla ayuda al usuario a no equivocarse.

10.5 ¿Por qué pusieron la lógica en la base de datos y no solo en el programa?

Porque el programa es solo una de las formas de llegar a los datos. Si las reglas viven únicamente en las pantallas, cada pantalla nueva tiene que recordar todas las reglas, y cualquier error u olvido deja pasar datos incorrectos. Si viven en la base, se escriben una vez y se cumplen siempre, venga el cambio de donde venga.

Un ejemplo claro: dar de baja una unidad exige un motivo de al menos cinco caracteres. Esa regla está en cuatro procedimientos distintos porque hay cuatro caminos que pueden terminar en una baja (cambiar estado, mantenimiento, devolución y el cambio individual). Si estuviera solo en una pantalla, los otros tres caminos se la saltarían.

Hay una segunda razón, de mantenimiento. La pantalla puede rediseñarse por completo, como pasó durante el proyecto, y las reglas de negocio no se tocan porque no dependen de ella.

Para ser precisos: no todo está en la base. Las validaciones rápidas, como un campo vacío, se hacen también en la pantalla para dar un aviso inmediato junto al campo. La diferencia es que la pantalla da comodidad y la base da garantía.

10.6 ¿Cómo se instala la base de datos y qué pasa si ejecutan el script dos veces?

Se abre `BaseDeDatos_Completa.sql` en SQL Server Management Studio y se ejecuta completo. Crea la base si no existe, luego las tablas, funciones, vistas, procedimientos, triggers y datos.

Si se ejecuta otra vez no rompe nada ni borra datos. Cada tabla se crea solo si no existe, con `IF OBJECT_ID(...) IS NULL`. Las vistas, funciones, procedimientos y triggers usan `CREATE OR ALTER`, que los crea o los actualiza sin fallar. Los datos iniciales y de ejemplo se insertan solo si faltan. Esa propiedad, que repetir la operación da el mismo resultado, se llama idempotencia.

También incluye una guardia: si la base no se puede crear o abrir, el script se detiene en lugar de seguir ejecutándose sobre otra base por error. Y el script no crea usuarios: el primer administrador se crea desde el asistente de primer uso del programa, para que su contraseña nunca viaje en un archivo.

10.7 ¿Qué diferencia hay entre dar de baja y eliminar? ¿Por qué casi no borran filas?

Eliminar es destruir la fila para siempre. Dar de baja es marcarla como inactiva y conservarla. En este sistema se prefiere la baja lógica porque casi todo tiene historia que importa: un empleado que salió sigue apareciendo en los pagos que recibió, y una unidad dada de baja sigue apareciendo en los préstamos que tuvo. Si se borraran, esos registros quedarían apuntando a nada, o se perdería el historial.

Lo hacen de formas distintas según el caso. El empleado pasa al estado 2, Inactivo. El pago y la ausencia se anulan con una bandera (`Anulado`, `Anulada`) y sus índices ignoran los anulados. La unidad cambia a "Dado de baja" y conserva la fecha y el motivo. El usuario se desactiva con `Activo = 0`. Y el salón se desactiva en lugar de eliminarse.

Lo que sí se borra de forma física son cosas que no tienen historia propia, y el historial mismo es lo contrario: un trigger impide borrarlo.

10.8 ¿Cómo funciona el primer uso del sistema?

La primera vez que se abre el programa, la base existe pero está vacía de configuración y de usuarios. El programa detecta que no hay configuración y muestra un asistente donde se piden los datos de la institución y los del primer administrador, incluyendo su correo, que se verifica con un código enviado por correo.

Cuando se confirma, la aplicación convierte la contraseña en un hash BCrypt y llama a `sp_GuardarConfiguracionInicial`. Ese procedimiento, dentro de una transacción, guarda la configuración y crea el usuario con rol Administrador. Si el sistema ya estaba configurado, lanza el error 50021, así que nadie puede ejecutar el asistente de nuevo para crearse un administrador. Además, la tabla `ConfiguracionEmpresa` tiene un `CHECK` que obliga a que su única fila tenga `IdConfig = 1`: es imposible tener dos configuraciones.

11. Preguntas y respuestas, bloque 2: tablas, llaves y restricciones

Estas son las preguntas que un profesor de bases de datos haría apenas vea el modelo. Cada respuesta parte de una tabla real de tu script, para que puedas señalarla mientras explicas.

11.1 ¿Qué diferencia hay entre una llave primaria y una foránea, y cómo puede una misma columna ser las dos?

La llave primaria responde a la pregunta «¿qué fila eres?» y la foránea a «¿de qué otra fila dependes?». La primaria vive en la tabla dueña del dato y asegura dos cosas: que no existan dos filas con el mismo valor y que ninguna fila quede sin identificar. En `Empleados` es `IdEmpleado`: el motor rechaza un segundo empleado con el mismo número y también rechaza un `NULL`. La foránea vive en la tabla que quiere hacer referencia a ese dato: `Empleados.IdCargo` guarda un número, y el motor exige que ese número exista en `Cargos.IdCargo`.

El efecto práctico va en las dos direcciones. Hacia adelante, no puedes asignar a un empleado un cargo que no existe. Hacia atrás, no puedes borrar un cargo mientras algún empleado lo use, porque ese empleado quedaría apuntando a nada; a eso se le llama fila huérfana. Lo importante es quién lo impide: el motor, no el programa. Por eso la regla se cumple aunque alguien escriba SQL directo desde Management Studio.

Que una columna sea las dos cosas no es contradictorio, porque cada llave habla de algo distinto: la primaria habla de unicidad y la foránea de existencia. El caso más claro es `EmpleadoFotos`: su llave primaria es `IdEmpleado` (no puede haber dos fotos para el

mismo empleado) y a la vez es foránea hacia `Empleados` (la foto no puede pertenecer a un empleado que no existe). Juntas expresan una relación uno a uno. En `EmpleadoMaterias` pasa algo parecido, pero con dos columnas: la llave primaria es la pareja `(IdEmpleado, IdMateria)` y cada una de las dos es además foránea. Eso se desarrolla en la pregunta 11.9.

Un detalle que ayuda al explicar: todas las restricciones tienen nombre con prefijo, `PK_` para primaria, `FK_` para foránea, `UQ_` para única, `CK_` para chequeo y `DF_` para valor por defecto. Cuando SQL Server rechaza una operación, ese nombre aparece en el mensaje de error y te dice enseguida qué regla se violó y en qué tabla.

11.2 ¿Para qué sirve IDENTITY y por qué `Salones` no lo usa?

`IDENTITY(1,1)` le dice al motor que genere el número él mismo: empieza en 1 y suma 1 con cada fila nueva. Casi todas tus tablas lo usan (`Empleados`, `Cargos`, `Prestamos`, `PagosPlanilla`…) porque libera al programa de inventar identificadores, y lo hace de forma segura cuando dos personas guardan al mismo tiempo: el motor entrega un número distinto a cada una sin que se pisen. Hay dos cosas que conviene saber. La primera es que el número no se reutiliza: si borras la fila 7, la siguiente sigue siendo la 8, y si una transacción falla puede consumir un número y dejar un hueco, así que nunca debes suponer que los identificadores son consecutivos. La segunda es que no se puede insertar un valor propio en esa columna sin pedir un permiso especial (`SET IDENTITY_INSERT`).

`Salones` es la excepción: `IdSalon` es `INT NOT NULL` sin `IDENTITY`, y `sp_GuardarSalon` calcula el siguiente con `ISNULL(MAX(IdSalon), 0) + 1`. Es hacer a mano lo que `IDENTITY` hace solo, y por eso necesita protección. Piensa en dos administradores que guardan un salón al mismo instante. Sin bloqueos, los dos leerían que el máximo es 5, los dos calcularían 6 y el segundo `INSERT` fallaría por llave duplicada. El procedimiento evita eso abriendo una transacción y leyendo el máximo con `UPDLOCK, HOLDLOCK`: el primero retiene el rango leído hasta confirmar, el segundo espera su turno, y cuando le toca ya ve el 6 y toma el 7.

Si te preguntan por qué no usaron `IDENTITY`, la respuesta honesta no es defender que el `MAX + 1` sea mejor. `IDENTITY` es más simple y más rápido. El `MAX + 1` es un rasgo del diseño de esa tabla que se conservó y se compensó con el bloqueo para que sea correcto. Cambiarlo es una mejora natural, aunque no trivial: SQL Server no permite convertir una columna ya existente en `IDENTITY` con un `ALTER`, así que habría que recrear la tabla y sus relaciones.

11.3 ¿Qué es un índice único filtrado y por qué lo usan en vez de un `UNIQUE` normal?

Un índice único normal dice «no puede haber dos filas con el mismo valor en esta columna». Uno filtrado añade un `WHERE` y dice «no puede haber dos filas con el mismo valor, **contando solo las filas que cumplan este filtro**». Tu script lo usa porque hay reglas del tipo «solo una, mientras esté vigente», y un `UNIQUE` común no sabe de vigencias.

El primer caso es el DUI. Es opcional, así que muchos empleados tienen `NULL`. En SQL Server un `UNIQUE` normal trata los `NULL` como si fueran iguales entre sí, de modo que solo permitiría un empleado sin DUI. `UX_Empleados_DUI` declara `WHERE DUI IS NOT NULL`: los vacíos no cuentan, y los que tienen valor no se pueden repetir.

El segundo caso es el más elegante: `UX_Prestamos_UnidadActiva` es único sobre `IdUnidad` pero solo para filas con `Estado = 'Activo'`. Una pieza física no puede estar prestada dos veces a la vez, pero a lo largo de su vida puede aparecer en muchos préstamos ya devueltos, y eso es historial legítimo. Un `UNIQUE` normal prohibiría el segundo préstamo de la historia de la unidad. Además es una red de seguridad: `sp_RegistrarPrestamo` ya comprueba que la unidad esté disponible, pero si dos personas pulsan a la vez, el índice es lo que impide que entren los dos.

El tercero conecta con el borrado lógico: `UX_PagosPlanilla_Empleado_Periodo` es único sobre `(IdEmpleado, Periodo)` solo donde `Anulado = 0`. Hay un solo pago vigente por empleado y mes; si lo anulas, deja de contar y se puede generar uno nuevo. La «eliminación» es una bandera, y el índice entiende la bandera. Con la misma idea, `UX_ReportesDanio_UnidadAbierta` permite un solo reporte abierto por unidad, y los códigos de inventario son únicos solo cuando existen.

Cuando uno de estos índices se viola, SQL Server devuelve el error 2601; cuando se viola una restricción `UNIQUE` (como `UQ_Empleados_Nombre`) devuelve el 2627. Por eso el código de las pantallas atiende los dos números para mostrar el aviso de «ya existe». Aparte hay un índice normal, `IX_UnidadesActivo_SalonActivo`, que no impone ninguna regla: solo acelera la búsqueda de unidades por salón.

11.4 ¿Cuál es la diferencia entre un `CHECK`, un trigger y una validación en C#? ¿Cuándo se usa cada uno?

Se distinguen por lo que pueden ver y por lo que pueden hacer. Un `CHECK` es una regla que mira únicamente la fila que se está guardando: `CK_Salones_Capacidad` exige que la capacidad esté entre 1 y 500, y `CK_PagosPlanilla_Montos` compara columnas de la

misma fila (que el descuento no supere el salario base más el bono). No puede consultar otras tablas ni otras filas. A cambio es instantáneo, no se puede olvidar y no se puede esquivar: aplica a cualquier `INSERT` o `UPDATE`, venga de donde venga.

Un detalle que sorprende: un `CHECK` solo rechaza cuando la condición da falso. Si la columna es `NULL`, la condición da «desconocido» y la fila pasa. Por eso `DUI` puede quedar vacío aunque tenga un patrón, y por eso, para exigir que un dato exista, hay que declarar la columna `NOT NULL`, no usar un `CHECK`.

Cuando la regla necesita mirar otras filas o tablas, un `CHECK` no alcanza, y tu sistema tiene un ejemplo exacto: el tope de 40 horas semanales de un docente. La tabla `AsignacionesDocente` tiene un `CHECK` que exige que cada asignación tenga entre 1 y 40 horas, pero el tope real es sobre la suma de todas las materias del docente en el año, y sumar otras filas es algo que un `CHECK` no puede hacer. Eso lo valida `sp_AsignarDocente` (error 50204: «Supera el máximo de 40 horas semanales: el docente ya tiene N h asignadas»). Igual pasa con «no se puede prestar una unidad en mal estado, en reparación o dada de baja»: exige consultar el estado de la unidad, y lo hace `sp_RegistrarPrestamo` (error 50034). Los triggers cumplen otra función: no validan datos de entrada, reaccionan a cambios ya hechos, como escribir el historial de auditoría o impedir que se edite.

La validación en C# (el campo vacío, el correo con forma de correo) es comodidad, no seguridad: da el aviso inmediato junto al campo, pero cualquiera que llegue a la base sin pasar por el formulario se la salta. La regla práctica para explicarlo: lo que debe ser cierto siempre va en la base; lo que ayuda al usuario va además en la pantalla. Y hay una razón extra para tener ambas capas: el mensaje de un `CHECK` fallido es técnico («conflicto con la restricción CK_...»), mientras que un procedimiento puede lanzar un texto claro con `THROW`. Por eso las reglas importantes suelen tener una versión amable en el procedimiento y la red de seguridad en la tabla.

11.5 ¿Qué hace `ON DELETE CASCADE` y por qué solo está en cinco relaciones?

Por defecto una llave foránea protege: si intentas borrar la fila padre mientras haya hijas que la usan, el motor cancela el borrado. `ON DELETE CASCADE` invierte esa conducta: al borrar el padre, el motor borra solo las hijas. Lo usas en cinco relaciones: `EmpleadoFotos`, `EmpleadoMaterias` y `AsignacionesDocente` respecto a `Empleados`, y `UnidadesActivo` y `ReportesDanio` respecto a `GestionActivos`.

Lo que tienen en común esas cinco es que las filas hijas no tienen sentido propio sin el padre. Una foto sin empleado, o la lista de materias que puede dar un docente que ya no existe, no le sirven a nadie. Mantenerlas obligaría a borrarlas a mano antes del padre, y

olvidarlo sería un error.

Lo revelador es dónde no está. `PagosPlanilla` y `Prestamos` también apuntan a `Empleados` y a `UnidadesActivo`, pero sin cascada, porque son historial y el historial se conserva. Eso tiene una consecuencia que explica el sistema entero: un empleado con pagos registrados no se puede borrar físicamente, el motor lo impide. Esa es la razón de fondo de que el programa lo dé de baja (estado Inactivo) en lugar de eliminarlo.

La cascada es poderosa y por eso peligrosa: un solo `DELETE` puede arrastrar muchas filas sin avisar. Pero el motor evalúa toda la cadena y, si una sola fila queda protegida por otra llave sin cascada, cancela la operación completa. Por ejemplo, borrar un tipo de activo arrastraría sus unidades, pero si alguna de esas unidades tuvo préstamos, `Prestamos` la protege y el borrado entero falla. Para defenderlo: la cascada se usa solo para datos dependientes sin historia propia; para todo lo demás, la ausencia de cascada es lo que fuerza la baja lógica.

11.6 ¿Qué es una columna calculada persistida y por qué `SalarioNeto` es una en `PagosPlanilla`?

En `PagosPlanilla` la columna se declara así: `SalarioNeto AS (SalarioBase + Bono - Descuento) PERSISTED`. No se escribe nunca: es el motor quien la calcula a partir de las otras tres. Si intentas hacer un `INSERT` o un `UPDATE` sobre ella, falla. `PERSISTED` significa que el resultado se guarda físicamente en la tabla en lugar de recalcularse cada vez que alguien lo consulta, y el motor lo actualiza solo cuando cambia alguna de las columnas de las que depende.

La razón de ser es evitar una inconsistencia. Si el neto fuera una columna normal, algún procedimiento, pantalla o consulta manual podría guardar uno que no coincida con la suma. Con la columna calculada, la contradicción es imposible: el neto es siempre base más bono menos descuento. Es una regla de negocio convertida en estructura. Y como el descuento incluye la parte por ausencias, cuando `sp_RegistrarAusencia` recalcula los descuentos de un pago pendiente, el neto se corrige solo, sin que nadie tenga que acordarse.

Fíjate en que `Planillas` tiene también una columna `SalarioNeto`, pero allí es una columna normal que se llena a mano (el script de datos la siembra como base menos descuento más bono). Es una diferencia real del diseño, y si te la señalan, puedes reconocerla con naturalidad: en el catálogo el neto es informativo, y en los pagos, que son el dinero que de verdad se entrega, está blindado. Convertir también la del catálogo en calculada sería una mejora coherente.

11.7 ¿Por qué hay una tabla `Planillas` y otra `PagosPlanilla`? ¿No es duplicar información?

Son dos cosas distintas que se parecen en las columnas. `Planillas` es el **catálogo de estructuras salariales**: cada fila es una plantilla («salario base de tanto, con tal descuento y tal bono») y los empleados apuntan a una con `Empleados.IdPlanilla`. `PagosPlanilla` es el **registro de lo que se pagó realmente**: una fila por empleado y por mes, con su periodo, su estado (Pendiente o Pagado), su fecha de pago y si fue anulado.

La copia de los montos (`SalarioBase`, `Bono`, `Descuento` aparecen en las dos) es intencional, no descuido. Cuando se genera la planilla de un mes, los valores de la estructura se copian al pago como una fotografía de ese momento. Imagina que en marzo se paga a un docente según su estructura vigente y en abril se sube el salario de esa estructura. Si el pago de marzo solo guardara una referencia a la estructura, al consultarlo en mayo mostraría el salario de abril y el historial quedaría falseado. Al copiar los números, marzo queda congelado. Es la misma idea de las facturas, que guardan el precio del día y no un enlace al precio actual.

El pago además tiene sus propias reglas, que son las que más se pueden señalar en una defensa. `CK_PagosPlanilla_Periodo` exige que el periodo sea siempre el día 1 de un mes, para que «abril» tenga una sola representación. `CK_PagosPlanilla_Estado` limita el estado a Pagado o Pendiente. Y `CK_PagosPlanilla_Montos` pide un salario base mayor que cero, bono y descuento no negativos, y descuento no mayor que base más bono, de modo que el neto nunca sea negativo. Junto con el índice único filtrado de la pregunta 11.3, un mismo mes no puede cobrarse dos veces a la misma persona.

11.8 ¿Por qué escogieron esos tipos de dato (`DECIMAL`, `VARCHAR`, `DATE`, `BIT`…)?

Cada tipo es una decisión sobre qué dato es. El dinero va en `DECIMAL(10,2)`: hasta 8 dígitos enteros y exactamente 2 decimales. Se evita `FLOAT` porque guarda aproximaciones binarias; con él, sumar 0.1 y 0.2 puede dar 0.30000000000000004, y en una planilla eso se traduce en centavos que no cuadran. `DECIMAL` es exacto.

Los identificadores como el DUI o el teléfono se guardan como texto (`VARCHAR`) aunque tengan dígitos. No son cantidades: no se suman ni se comparan por tamaño, pueden empezar con cero y el DUI lleva un guion. Por eso `CK_Empleados_DUI` revisa el patrón de ocho dígitos, guion y un dígito. Los textos tienen un largo máximo distinto según su uso (50 para un nombre de salón, 300 para una observación) para acotar lo que cabe y dar un

aviso claro cuando algo es demasiado largo. La mayoría usa `VARCHAR`; unas pocas columnas de `Usuarios` usan `NVARCHAR`, que guarda Unicode completo a costa de ocupar el doble. Para español con tildes y eñes, `VARCHAR` es suficiente.

Las fechas tienen dos tipos y la elección importa. `DATE` guarda solo el día: nacimientos, contratación, periodos. `DATETIME` guarda día y hora: la entrega de un préstamo, el registro de un pago o la auditoría. Usar `DATE` donde no hay hora evita comparaciones que fallan por unos minutos. Los verdadero/falso (`Activo`, `Anulado`, `CorreoVerificado`) son `BIT`, que solo admite 0 y 1. Y las fotos son `VARBINARY(MAX)`, datos binarios sin límite práctico, en una tabla aparte (`EmpleadoFotos`) para que las consultas normales de empleados no arrastren imágenes pesadas.

11.9 ¿Cómo se representa que un docente da varias materias y una materia la dan varios docentes?

Es una relación muchos a muchos, y las bases relacionales no pueden guardarla directamente en dos tablas: ni `Empleados` puede tener una lista de materias en una celda, ni `Materias` una lista de docentes. La solución es una tabla intermedia, `EmpleadoMaterias`, donde cada fila es un hecho: «este empleado puede dar esta materia». Tiene solo dos columnas, `IdEmpleado` e `IdMateria`, cada una con su llave foránea. La llave primaria es la pareja de ambas, y eso cumple un segundo trabajo: impide registrar dos veces la misma combinación, porque la pareja no puede repetirse.

En el sistema hay otra tabla del mismo estilo pero con más información, `AsignacionesDocente`, que es la carga académica. Conviene que no las confundas, porque se pregunta mucho. `EmpleadoMaterias` dice lo que el docente **puede** dar (su habilitación). `AsignacionesDocente` dice lo que **da** de hecho: con qué sección, en qué año lectivo y cuántas horas semanales. Esta última tiene su propio `IdAsignacion`, una restricción única sobre `(IdEmpleado, IdSeccion, IdMateria, AnioLectivo)` para que no se asigne dos veces lo mismo, y un `CHECK` que limita las horas a un rango entre 1 y 40. La primera puede entenderse como el permiso, y la segunda como el horario concreto.

Cuando asignas una carga, `sp_AsignarDocente` consulta justamente estas dos tablas. Verifica que el empleado tenga un cargo de docente, que la sección exista y que la materia esté entre las que ese docente puede dar en `EmpleadoMaterias` (si no indicas materia y el docente tiene una sola, usa esa; si no tiene ninguna, avisa que hay que registrarlas en su ficha de personal). Después revisa que no se repita la misma asignación y que la suma de

horas no pase de 40. Las dos tablas conversan a través de esas reglas. Una pista para razonar cualquier otro caso: cada vez que veas una tabla cuya llave primaria es una pareja de llaves foráneas, es casi seguro que representa una relación muchos a muchos.

11.10 ¿Qué pasa si el usuario intenta borrar algo que otra tabla está usando? ¿Quién lo detiene?

Hay dos defensas, y entender por qué hay dos es una buena respuesta. Toma el catálogo de estructuras salariales. En `frmCatalogoSalarios`, antes de borrar, el programa cuenta cuántos empleados tienen esa estructura (`SELECT COUNT(*) FROM Empleados WHERE IdPlanilla = ...`). Si hay alguno, no intenta borrar y muestra un mensaje claro: «no se puede eliminar porque está asignada a N empleados, incluidos los dados de baja». Esa es la defensa amable: la pantalla anticipa el problema y explica qué hacer.

Pero si esa comprobación se saltara, por un error o porque otra persona asignó la estructura un segundo antes de que se pulsara el botón, el `DELETE` llegaría a la base y chocaría con `FK_Empleados_Planilla`. El motor lo cancela con el error 547 (conflicto con una llave foránea). El código C# lo atrapa y lo traduce a «La estructura está en uso y no se puede eliminar». Esa es la defensa dura: ocurre aunque la pantalla falle, aunque el dato se toque desde otro programa, aunque dos usuarios se crucen.

La lección que puedes llevarte para la defensa es que la integridad no depende de que el programador no se equivoque. La pantalla se ocupa de la experiencia, y la base se ocupa de la verdad. Mira también el matiz de «incluidos los dados de baja»: los empleados inactivos siguen en la tabla y siguen apuntando a su estructura, justo porque la baja es lógica; si fueran borrados, esa referencia desaparecería. Dos decisiones del diseño, la baja lógica y la llave foránea, trabajan juntas.

12. Preguntas y respuestas, bloque 3: vistas y funciones

Las vistas y las funciones son lo que más se presta a la confusión, porque las dos «devuelven algo» y las dos se usan dentro de un `SELECT`. Las respuestas siguientes se detienen en esa diferencia y la ilustran con las piezas reales de tu base.

12.1 ¿Qué es una vista? ¿Guarda datos como una tabla?

No guarda nada. Una vista es una consulta con nombre: el motor memoriza el texto del `SELECT` y, cada vez que alguien escribe `SELECT * FROM vw_Prestamos`, lo ejecuta de nuevo sobre las tablas reales y entrega el resultado. Por eso siempre está al día: no existe

una copia que pueda quedar desactualizada. Si hoy se registra un préstamo, la vista lo muestra en el mismo instante, porque lo que lee es la tabla `Prestamos`.

Lo que sí guarda la base es la definición, y por eso puedes usarla como si fuera una tabla: filtrarla con `WHERE`, ordenarla, unirla con otras. Lo hace el programa a diario: `Datos.Tabla("SELECT * FROM vw_Prestamos WHERE Alerta = 'Atrasado'")` es una consulta normal que casualmente apunta a una vista.

Un error frecuente es creer que una vista es más rápida que escribir el `SELECT` a mano. No lo es: ejecutarla cuesta lo mismo que ejecutar su consulta interna. Su valor no está en la velocidad sino en la organización: una consulta compleja se escribe una vez, se prueba una vez y se reutiliza desde cualquier lugar. (Existen las «vistas indexadas», que sí guardan el resultado y son más rápidas, pero tu sistema no las usa y no hace falta mencionarlas más allá de saber que existen.)

12.2 Si C# puede hacer los `JOIN` por su cuenta, ¿para qué crear tantas vistas?

Porque si cada pantalla escribe sus propios `JOIN`, las mismas uniones quedan copiadas en diez lugares y cualquier cambio hay que repetirlo en los diez. Mira `vw_UnidadesDetalle`: para mostrar una unidad con su nombre de equipo, su categoría, su ubicación y su estado hay que unir cinco tablas (`UnidadesActivo`, `GestionActivos`, `Categorias`, `Salones` y `Estados`). Sin la vista, cada pantalla que muestre unidades repetiría esas cinco uniones. Con la vista, cada una dice `SELECT ... FROM vw_UnidadesDetalle` y listo.

Hay tres ventajas más. La primera es que la vista traduce números a palabras: la tabla guarda `IdEstado = 3`, la vista entrega `Estado = 'Malo'`, y la pantalla nunca tiene que saber qué significa el 3. La segunda es que permite calcular columnas una sola vez, como la `Alerta` de los préstamos, de manera que todas las pantallas coinciden en lo que significa «atrasado». La tercera es que si el día de mañana cambia el diseño (por ejemplo, se añade otra tabla para guardar proveedores), se corrige la vista y las pantallas que la consumen siguen funcionando sin tocarse.

La regla mental para explicarlo: las tablas están organizadas para guardar sin repetir; las vistas están organizadas para leer cómodamente. Se separan porque lo que conviene al guardar (datos dispersos en muchas tablas, con números) no es lo que conviene al mostrar (todo junto, con palabras).

12.3 ¿Por qué algunas vistas usan `JOIN` y otras `LEFT JOIN`? ¿Qué cambia?

La diferencia es qué pasa con las filas que no tienen pareja. Un `JOIN` normal (llamado `INNER`) solo devuelve las filas que encuentran pareja en la otra tabla; las que no, desaparecen del resultado. Un `LEFT JOIN` conserva todas las filas de la tabla de la izquierda y rellena con `NULL` lo que no encuentra.

En `vw_SalonesResumen` se ve la razón con claridad. La vista cuenta cuántas unidades tiene cada salón, y para eso une `Salones` con `UnidadesActivo`. Si usara `JOIN`, un salón recién creado, que todavía no tiene ningún equipo, no aparecería en la lista: desaparecería justo porque no tiene pareja. Con `LEFT JOIN` aparece con 0 unidades, que es lo correcto. Fíjate también en `COUNT(u.IdUnidad)` y no `COUNT(*)`: contar una columna ignora los `NULL`, así que un salón sin unidades da 0; con `COUNT(*)` daría 1, porque la fila del salón existe aunque no tenga unidades.

La regla de decisión es preguntarse: «¿puede existir esta fila sin la otra?». En `vw_UnidadesDetalle` el salón, el estado y el equipo son obligatorios (las columnas son `NOT NULL`), así que se usan `JOIN`; la categoría puede ser nula, así que lleva `LEFT JOIN`. Y en `vw_EmpleadosDetalle`, un empleado puede no tener cargo, ni estructura salarial, ni pago del mes: todos esos `LEFT JOIN` evitan que desaparezca de la lista de personal por falta de un dato. Para el pago del mes se usa además `OUTER APPLY` con `TOP 1`, una variante que permite traer «el último pago vigente de este empleado en el mes actual» y que, si no existe, devuelve vacío; por eso la vista muestra `Sin registrar` con un `ISNULL`.

12.4 ¿Cómo funcionan las alertas de `vw_Prestamos` y `vw_StockConsumibles`?

Son columnas calculadas con `CASE`, que equivale a una cadena de «si… si no…». El motor evalúa las condiciones de arriba hacia abajo y se queda con la primera que se cumple. En `vw_Prestamos` el orden es: si el préstamo ya fue devuelto, `Devuelto`; si no y la fecha límite ya pasó, `Atrasado`; si no y falta menos de dos días, `Por vencer`; y si nada de eso, `En curso`. El orden importa por una razón concreta: un préstamo devuelto hace un mes tiene la fecha límite vencida, y si la pregunta del atraso fuera primero, aparecería como atrasado para siempre. Por eso `Devuelto` se pregunta primero.

La vista usa `GETDATE()`, la hora actual del servidor, y como se evalúa cada vez que se consulta, la alerta cambia sola con el paso del tiempo sin que nadie actualice nada. Un préstamo que hoy está `En curso` mañana puede ser `Por vencer` y pasado mañana `Atrasado`, sin que se toque ninguna fila. Es lo contrario de guardar la alerta como una columna: una columna guardada habría que recalcularla todos los días.

`vw_StockConsumibles` sigue la misma idea con el inventario: `Agotado` si el stock es 0, `Stock bajo` si es menor o igual al mínimo, `Normal` en otro caso. Lo interesante es cómo lo aprovecha el programa. La pantalla de préstamos filtra con `WHERE Alerta = 'Atrasado'`, ordena con un `CASE` para poner primero lo urgente, cuenta los atrasados y los por vencer para el aviso del encabezado, y pinta cada fila de rojo o ámbar según el valor de `Alerta`. Todo eso gira alrededor de una definición única hecha en la base. Si mañana el negocio decide que «por vencer» es a tres días y no a dos, se cambia un número en la vista y las cuatro funciones de la pantalla lo respetan.

12.5 ¿Cómo calcula `vw_StockBodega` cuántas unidades están disponibles?

Cuenta las unidades que cumplen tres condiciones a la vez, y cada una está en una parte distinta de la consulta. La primera: estar en la bodega, con `WHERE u.IdSalon = dbo.fn_IdBodega()`. La segunda: no tener un estado problemático, con `e.NombreEstado NOT IN ('Malo', 'En reparación', 'Mantenimiento Pendiente', 'Dado de baja')`. Una unidad en estado `Regular` sí cuenta como disponible: no está estropeada, solo no es nueva. La tercera: no estar prestada en este momento, con `NOT EXISTS (SELECT 1 FROM Prestamos p WHERE p.IdUnidad = u.IdUnidad AND p.Estado = 'Activo')`.

`NOT EXISTS` merece una explicación. Es una pregunta de ausencia: «no existe ningún préstamo activo para esta unidad». Es la forma más limpia de excluir filas según lo que *no* hay en otra tabla, y funciona bien con `NULL`. Con una unión normal habría que buscar los que sí tienen préstamo y restarlos, que es más enrevesado. Después `GROUP BY` agrupa por tipo de equipo y `COUNT` cuenta, de manera que el resultado es «tantos pupitres, tantas laptops, listos para asignar».

Se nota la ventaja de poner esto en una vista: la pantalla de asignación solo pregunta «¿cuántos hay disponibles?» y no necesita saber qué es un estado problemático ni cómo se ve un préstamo activo. Si mañana aparece un estado nuevo, como «En garantía», se edita la lista de la vista y todo el sistema se adapta.

12.6 ¿Cómo se construye el registro de actividad legible (`vw_Auditoria`) a partir de datos en JSON?

La tabla `Auditoria` guarda, por cada cambio, tres cosas crudas: qué tabla, qué operación (I, U, D) y dos textos JSON, `Antes` y `Despues`, con el contenido de la fila. Eso es completo pero ilegible para una persona. `vw_Auditoria` lo convierte en frases.

Primero traduce el nombre técnico de la tabla a un módulo comprensible, con un `CASE`: `PagosPlanilla` pasa a «Pagos de planilla» y `UnidadesActivo` a «Activos (unidades)». Después decide la acción, otra vez con un `CASE` ordenado de lo específico a lo general. Mira el patrón: si la operación es una modificación y `Anulado` pasó de `false` a `true`, la acción es «Anuló»; si `EstadoPago` pasó de `Pendiente` a `Pagado`, «Marcó como pagado»; si un usuario pasó de activo a inactivo, «Desactivó». Para leer dentro del JSON usa `JSON_VALUE(a.Despues, '$.Anulado')`, que extrae un campo del texto. Solo si ninguna regla específica aplica cae en los genéricos: `I` es «Creó», `D` es «Eliminó» y todo lo demás «Modificó». Es el mismo principio de la alerta de préstamos: lo más específico primero.

Para el resumen usa `STRING_AGG` sobre la función `fn_AuditoriaCambios`, que devuelve una fila por campo modificado, y las pega separadas por punto y coma, cortadas a 600 caracteres para que la tabla de la pantalla no se desborde. El resultado se lee así: «SalarioBase: 600.00 → 650.00; Bono: 0.00 → 25.00». Por último, hace un `LEFT JOIN` con `Usuarios` para mostrar el nombre completo, y si la fila no tiene usuario muestra `(sistema)`, que ocurre cuando alguien cambia datos directamente desde Management Studio.

12.7 ¿Cuál es la diferencia entre una vista, una función y un procedimiento almacenado?

Las tres son código guardado en la base, pero responden a preguntas distintas. Una **vista** no recibe parámetros: es una consulta fija a la que luego se le pueden poner filtros desde afuera. Una **función** recibe parámetros y devuelve un valor (un número, una fecha, un sí/no) o una tabla, y está pensada para usarse *dentro* de otra consulta: `WHERE u.IdSalon = dbo.fn_IdBodega()`. Un **procedimiento** también recibe parámetros, pero se invoca por sí mismo con `EXEC`, puede ejecutar varias instrucciones, abrir transacciones, modificar datos y lanzar errores.

Lo que una función no puede hacer es modificar tablas: no puede insertar, actualizar ni borrar datos permanentes. Eso es una garantía a favor, porque se puede usar en cualquier consulta con la seguridad de que solo calcula y no cambia nada. Los procedimientos, que sí cambian cosas, son los que llevan las transacciones.

Tu script tiene seis funciones de dos clases. Cinco son *escalares* (devuelven un único valor): `fn_DiasHabiles`, `fn_DiasDescontados`, `fn_DescuentoAusencias`, `fn_PeriodoCerrado` y `fn_IdBodega`. Una es *de tabla en línea*: `fn_AuditoriaCambios`. Esa última se parece mucho a una vista, con una diferencia crucial: recibe un parámetro

(`@IdAuditoria`). Por eso es una función y no una vista; una vista no puede preguntar «dame los cambios de *esta* fila de auditoría». Una buena forma de recordarlo: una función de tabla en línea es una vista con parámetros.

12.8 ¿Cómo cuenta `fn_DiasHabiles` los días de lunes a viernes y por qué no usa `DATEPART(WEEKDAY, ...)`?

La idea es de aritmética. Cualquier grupo de siete días seguidos contiene exactamente cinco días hábiles, sin importar con qué día empiece. Entonces la función calcula cuántos días hay en total (`DATEDIFF(DAY, @Ini, @Fin) + 1`), divide entre 7 para saber cuántas semanas completas caben y multiplica por 5. Lo que sobra, entre 0 y 6 días sueltos, se recorre uno por uno comprobando si cae en lunes a viernes.

Para saber qué día de la semana es cada día suelto, usa un truco: el 1 de enero de 1900 fue lunes, así que `DATEDIFF(DAY, '19000101', @Ini) % 7` da 0 si la fecha es lunes, 1 si es martes… y 6 si es domingo. Con ese número, `< 5` significa lunes a viernes. Un ejemplo para comprobarlo: del lunes 5 al miércoles 21 de octubre de 2026 hay 17 días. Son 2 semanas completas (2 × 5 = 10) y sobran 3 días, que son lunes, martes y miércoles, los tres hábiles, así que el total es 13. Al contarlo a mano en el calendario: 5 + 5 + 3 = 13.

La razón de no usar `DATEPART(WEEKDAY, ...)` es que su resultado depende de una configuración del servidor (`DATEFIRST`) y del idioma de la sesión: en un servidor donde la semana empieza en domingo, el lunes sería 2 en vez de 1, y la función daría resultados distintos según dónde se instale el sistema. El truco de 1900 no depende de nada del entorno. Es una decisión defensiva para que la base se comporte igual en la computadora de quien presenta y en cualquier otra.

Una limitación que conviene admitir con naturalidad si te la preguntan: la función no conoce los feriados. Cuenta lunes a viernes aunque sea festivo. Para incorporarlos habría que añadir una tabla de feriados y restarlos.

12.9 ¿Cómo se calcula el descuento por ausencias? Explícalo con un ejemplo

Se reparte en dos funciones que se llaman en cadena. `fn_DiasDescontados` responde «¿cuántos días hábiles de este mes se le descuentan a este empleado?». Busca sus ausencias del mes que no estén anuladas y solo cuenta dos tipos: las faltas injustificadas y los permisos sin goce de sueldo. Una falta justificada o un permiso con goce no descuentan. Si una ausencia empieza el 28 de septiembre y termina el 3 de octubre, a octubre solo le toca la parte que cae en octubre: la función recorta cada ausencia a los límites del mes y llama a `fn_DiasHabiles` con ese tramo.

Después `fn_DescuentoAusencias` convierte esos días en dinero con una regla simple: el salario base se divide entre los días hábiles del mes para obtener lo que vale un día, y se multiplica por los días descontados. Con un salario de 600 en octubre de 2026, que tiene 22 días hábiles, un día vale 600 ÷ 22 = 27.27. Si el empleado tuvo 3 días descontables, el descuento es 27.27 × 3, es decir 81.82 después de redondear a dos decimales. Si el mes no tuviera días hábiles, la función devuelve 0 para evitar dividir entre cero.

Estas funciones se conectan con el resto. `sp_GenerarPlanilla` suma el descuento de la estructura salarial más el de ausencias para llenar `PagosPlanilla.Descuento`, y lo limita a que no supere el salario base más el bono con un `CASE`; sin ese límite, un empleado con muchas faltas haría violar el `CHECK` de montos y la generación fallaría. `sp_RegistrarAusencia`, por su parte, llama a `sp_RecalcularDescuentoAusencias`, que actualiza el descuento de los pagos pendientes de los meses que abarca la ausencia, con el mismo tope. No toca los pagos ya pagados, los anulados ni los de meses cerrados, y anular una ausencia dispara el mismo recálculo. Como el neto es una columna calculada, se corrige solo.

12.10 ¿Para qué sirven `fn_IdBodega` y `fn_PeriodoCerrado`, que son tan cortas?

Sirven para dar un nombre a una pregunta que se repite en muchos sitios, y las funciones cortas son la mejor evidencia del beneficio. `fn_IdBodega()` responde «¿cuál es el salón de la bodega?». Devuelve el id del salón activo de tipo `Bodega`, prefiriendo el que se llama `Bodega central` y, si hubiera varios, el de menor id. Sin la función habría que escribir `IdSalon = 1` en vistas y procedimientos, un número mágico que se rompe si alguien crea otra bodega o si el id cambia. Con la función, la bodega se *descubre* en vez de suponerse: la vista `vw_StockBodega` y los procedimientos `sp_AltaUnidades` y `sp_AsignarUnidades` la usan, y si mañana se renombra o se reemplaza la bodega, se cambia en un solo lugar.

`fn_PeriodoCerrado(@Fecha)` responde «¿el mes de esta fecha está cerrado?». Convierte cualquier fecha en el primer día de su mes con `DATEFROMPARTS(YEAR(@Fecha), MONTH(@Fecha), 1)`, busca esa fila en `PeriodosPlanilla` y devuelve 1 si está cerrada. Si el mes nunca se ha cerrado y no existe fila, el `ISNULL(..., 0)` hace que devuelva 0, que es lo correcto (un mes sin registro está abierto). Los procedimientos que generan, registran, marcan como pagado o anulan un pago (`sp_GenerarPlanilla`, `sp_RegistrarPago`, `sp_MarcarPagado`, `sp_AnularPago`) la llaman al empezar y, si devuelve 1, lanzan el error 50800 («El mes está cerrado») antes de tocar nada. `sp_CerrarPeriodo` y `sp_ReabrirPeriodo` la usan al revés, para no cerrar un mes que ya está cerrado ni reabrir uno que está abierto. Las ausencias aplican la misma regla con una

comprobación directa sobre `PeriodosPlanilla` (error 50801), y el recálculo de descuentos la usa para saltarse los meses cerrados. Es la pieza que hace cumplir una de las reglas de negocio más importantes: un mes cerrado es inmutable.

La lección general es que una función pequeña con un buen nombre vale más que un comentario. Quien lea `IF dbo.fn_PeriodoCerrado(@Periodo) = 1` entiende la intención sin abrir la función.

12.11 ¿Cómo encuentra `fn_AuditoriaCambios` qué campos cambiaron entre dos textos JSON?

Esta es la función más sofisticada del script y se explica en tres pasos. El primero: el trigger guardó dos textos, `Antes` y `Despues`, con la forma `{"SalarioBase": 600, "Bono": 0}` y `{"SalarioBase": 650, "Bono": 25}`. La función los abre con `OPENJSON`, que convierte un JSON en una tabla de dos columnas, `key` (el nombre del campo) y `value` (su valor).

El segundo paso es alinear las dos tablas por nombre de campo con un `FULL OUTER JOIN`. Se usa *full* porque un campo puede existir en un lado y no en el otro: en un alta no hay «antes» y en una baja no hay «después», y el `FULL OUTER JOIN` conserva las filas de ambos lados aunque no tengan pareja. Si en una alta no hubiera `FULL`, no habría nada que mostrar. Para los casos en que falta uno de los JSON usa `ISNULL(a.Antes, N'{}')`, un objeto vacío.

El tercer paso es quedarse solo con lo relevante: en una modificación (`U`) conserva únicamente los campos donde el valor anterior difiere del nuevo; en altas y bajas (`I` y `D`) devuelve todos. Luego arma una línea de texto por campo, con una flecha para los cambios: `SalarioBase: 600.00 → 650.00`. Es una función *de tabla en línea* porque devuelve varias filas, una por campo, y `vw_Auditoria` la consume con `STRING_AGG` para pegarlas en un resumen.

El diseño tiene una consecuencia valiosa: el historial no depende de las columnas de cada tabla. Si mañana se añade una columna `Telefono2` a `Empleados`, la auditoría la registrará y la función la mostrará sin cambiar una línea, porque trabaja con los nombres que vengan en el JSON.

12.12 ¿Qué limitaciones tienen las funciones y vistas de este diseño? ¿Hay vistas que no se usan?

La primera limitación es de rendimiento y se explica con una imagen: una función escalar se ejecuta *una vez por cada fila* que la usa. Cuando `sp_GenerarPlanilla` calcula el descuento de ausencias de cada empleado, llama a `fn_DescuentoAusencias`, que a su vez llama a `fn_DiasDescontados`, que llama a `fn_DiasHabiles` por cada ausencia. Con los volúmenes de una escuela (decenas o cientos de empleados) eso es imperceptible; con millones de filas sería un problema, y habría que reescribirlo para procesar todo el conjunto de una vez. Reconocerlo muestra criterio: el diseño es adecuado para la escala del sistema.

La segunda es que las vistas con `GETDATE()` (la alerta de préstamos, el pago del mes actual en `vw_EmpleadosDetalle`) dependen del reloj del servidor, no del de la computadora del usuario. En una instalación local son el mismo equipo, así que no hay diferencia, pero si la base estuviera en un servidor remoto, la hora que cuenta sería la del servidor.

Tercera: de las 27 vistas, siete son resúmenes que el programa no consume actualmente (`vw_ResumenActivosPorCategoria`, `vw_ResumenActivosPorEstado`, `vw_ResumenActivosPorSalon`, `vw_ResumenEstadoPorSalon`, `vw_ResumenEmpleadosPorEstado`, `vw_ResumenPlanillaPorCargo` y `vw_ActivosPorSalon`). Si te lo preguntan, lo honesto es decirlo así: son consultas de resumen listas para usar, por ejemplo para armar gráficos o reportes, o para que la página web del equipo las lea en el futuro, y se pueden mostrar directamente en Management Studio con un `SELECT *`. Que existan no cuesta nada, porque una vista que no se consulta no consume recursos. Lo que no conviene es afirmar que alimentan el panel del programa, porque no es así.

13. Preguntas y respuestas, bloque 4: procedimientos almacenados y transacciones

Aquí está el corazón de las reglas del sistema. Las respuestas se apoyan en cuatro procedimientos que conviene que conozcas casi de memoria porque ilustran todo lo demás: `sp_RegistrarMovimientoConsumible` (transacción y bloqueo), `sp_RegistrarMantenimiento` (varias escrituras que deben ir juntas), `sp_GenerarPlanilla` (vista previa y proceso masivo) y `sp_GuardarConfiguracionInicial` (el primer uso).

13.1 ¿Qué es un procedimiento almacenado y qué se gana frente a escribir el SQL dentro del programa en C#?

Un procedimiento almacenado es un programa pequeño escrito en T-SQL que vive dentro de la base de datos. Se llama «almacenado» porque se guarda con la base (con `CREATE OR ALTER PROCEDURE`) y se ejecuta con su nombre y sus parámetros: `EXEC dbo.sp_RegistrarPrestamo @IdUnidad = 5, ...`. Desde C# se invoca con `Datos.Procedimiento("sp_RegistrarPrestamo", Datos.P("@IdUnidad", id), ...)`, que arma el comando de tipo procedimiento y le pasa cada parámetro por separado.

Se gana, sobre todo, un lugar único para las reglas. Un préstamo exige que la unidad exista, que no esté en mal estado, que no esté ya prestada, que el responsable sea un empleado real y activo si es docente, y que la fecha límite sea futura. Si eso estuviera en la pantalla, cualquier otra pantalla, reporte o script que registre préstamos tendría que repetirlo y podría olvidarlo. En el procedimiento se escribe una vez y todos los caminos pasan por ahí.

Hay otras ventajas más técnicas. Los parámetros viajan separados del texto SQL, lo que cierra la puerta a la inyección SQL. El motor guarda el plan de ejecución y no lo recalcula en cada llamada. Se puede hacer que una operación de varios pasos sea atómica, es decir, que se haga completa o no se haga, sin que C# tenga que coordinar nada. Y la pantalla se vuelve simple: reúne los datos, llama al procedimiento y muestra el mensaje que vuelve.

Hay que ser honesto en un punto: no todo pasa por procedimientos. Las operaciones muy simples, como crear un cargo o editar un consumible, todavía usan un `INSERT` o un `UPDATE` con parámetros directamente desde C#. Eso es seguro, porque los parámetros están protegidos, pero no tiene las reglas en la base. La regla que puedes defender es que las operaciones con reglas de negocio o con más de una escritura pasan por procedimiento, y las altas y ediciones de un solo campo van directas.

13.2 ¿Qué es una transacción? Explícalo con un caso real del sistema

Una transacción es un grupo de operaciones que la base trata como una sola: o se completan todas, o no se completa ninguna. El ejemplo ideal es `sp_RegistrarMovimientoConsumible`, que registra la salida de, digamos, 50 hojas de papel bond. Eso exige dos escrituras: actualizar `Consumibles.StockActual` y agregar una fila al kardex (`MovimientosConsumible`) con el saldo resultante. Imagina que el programa se cierra, o se corta la luz, justo después de la primera y antes de la segunda. Sin transacción, el stock habría bajado pero no habría registro de por qué; el inventario y su historial se contradirían para siempre.

Con la transacción, `BEGIN TRANSACTION` abre el grupo, las dos escrituras se hacen, y `COMMIT` las confirma juntas. Si algo falla antes del `COMMIT`, el motor deshace todo (`ROLLBACK`) y la base queda exactamente como estaba. Eso es lo que significa la **A** de ACID, atomicidad. Las otras tres letras se resumen así: **C**onsistencia (la base pasa de un estado válido a otro válido, nunca por uno intermedio roto), a**I**slamiento (dos usuarios simultáneos no ven ni pisan el trabajo a medias del otro) y **D**urabilidad (lo confirmado sobrevive aunque se apague el equipo).

Otro caso muy claro es `sp_CerrarPeriodo`: cierra el mes en `PeriodosPlanilla` y escribe la acción en `PeriodosPlanillaHistorial`. Si pudiera quedar cerrado sin la fila del historial, perderías la prueba de quién lo cerró y cuándo. Y en `sp_RegistrarMantenimiento` son tres escrituras: el registro del servicio, el cambio de estado de la unidad y el cierre del reporte de daño. El procedimiento se ve bien en acción cuando el reporte ya estaba cerrado: el `THROW 50051` ocurre después de las dos primeras escrituras, y el `CATCH` hace `ROLLBACK`, de modo que el mantenimiento recién insertado y el cambio de estado de la unidad desaparecen. No queda nada a medias.

13.3 ¿Por qué algunos procedimientos tienen `BEGIN TRANSACTION` y otros no?

Porque una transacción explícita solo hace falta cuando hay más de una escritura que deben ir juntas. Una sola sentencia de `INSERT`, `UPDATE` o `DELETE` ya es atómica por sí misma: el motor garantiza que o cambia todas las filas que le tocan, o ninguna. `sp_RegistrarPrestamo` solo tiene un `INSERT` al final, así que no necesita transacción. `sp_CambiarEstadoUnidades` cambia el estado de muchas unidades a la vez con un único `UPDATE ... FROM ... JOIN`, y por eso, aunque se seleccionen veinte unidades, o se cambian las veinte o ninguna, sin necesidad de abrir una transacción.

También se puede ver cómo se organiza el interior de los procedimientos con transacción: casi todas las validaciones que no necesitan datos bloqueados van *antes* del `BEGIN TRANSACTION`. En `sp_RegistrarMantenimiento` se comprueban la descripción, la existencia de la unidad, el estado nuevo y las reglas de la baja, y solo después se abre la transacción para escribir. La razón es práctica: dentro de una transacción el motor mantiene bloqueos sobre lo que ha tocado, y mientras duran, otros usuarios pueden tener que esperar. Hay que mantenerla lo más corta posible: lo mínimo necesario y confirmar rápido.

Por eso, cuando te pregunten por qué hay unos con transacción y otros sin ella, la respuesta es de criterio, no de descuido: la transacción se pone donde protege algo, y se evita donde sería ruido.

13.4 ¿Qué hacen `SET NOCOUNT ON` y `SET XACT_ABORT ON`, que aparecen al inicio de casi todos los procedimientos?

`SET NOCOUNT ON` le dice al motor que no devuelva el mensaje «(3 filas afectadas)» después de cada sentencia. Parece trivial, pero esos mensajes son tráfico extra, y en algunos clientes pueden confundirse con resultados. Con él activado, lo único que sale del procedimiento es lo que el programador decide devolver con un `SELECT`, como el `SELECT @IdSalon AS IdSalon` de `sp_GuardarSalon`. Es una buena práctica estándar y no cambia la lógica.

`SET XACT_ABORT ON` es más importante. Por defecto, cuando una sentencia dentro de una transacción da un error en tiempo de ejecución, SQL Server a veces sigue con la siguiente instrucción y deja la transacción abierta, con la mitad hecho. Con `XACT_ABORT ON`, cualquier error de ejecución hace que toda la transacción quede marcada como imposible de confirmar: solo se puede deshacer. Esa es la red de seguridad que evita datos a medias. Por eso todos los bloques `CATCH` del script empiezan con `IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;`: preguntan si hay una transacción abierta (`@@TRANCOUNT` mayor que cero) y, si la hay, la deshacen.

Resumen para explicarlo: `NOCOUNT` ordena el ruido y `XACT_ABORT` garantiza que un error nunca deje una transacción a medias.

13.5 ¿Cómo funcionan `TRY/CATCH` y `THROW`? ¿Por qué el `CATCH` termina con un `THROW` sin parámetros?

`BEGIN TRY ... END TRY` envuelve el código que puede fallar. Si ocurre un error dentro, la ejecución salta de inmediato al bloque `BEGIN CATCH ... END CATCH`, que es donde se limpia: se deshace la transacción abierta. Pero deshacer no basta: si el `CATCH` se limitara a eso, el error se «tragaría» y la pantalla creería que todo salió bien. Por eso termina con `THROW;`, sin parámetros, que significa «vuelve a lanzar el mismo error que acabo de atrapar». El procedimiento limpia y además informa.

La otra forma de `THROW` es la que lleva parámetros: `THROW 50043, 'Stock insuficiente: no se puede despachar más de lo que hay.', 1;`. Son tres datos: un número de error propio, el mensaje y un estado. Los números propios deben ser mayores que 50000 para no chocar con los errores del propio SQL Server, y el script los agrupa por tema: 50031 a 50037 son de préstamos, 50040 a 50043 de consumibles, 50200 a 50207 de la carga académica, 50800 a 50808 del cierre de mes, y así. No es capricho: permiten saber de un vistazo de dónde viene un error y buscarlo en el código. El último comentario del script incluso lista estos códigos por procedimiento.

Lo valioso del mensaje es que está escrito para la persona que usa el sistema, no para el programador. C# no lo reescribe: el error llega como una `SqlException` y la pantalla muestra su propiedad `Message`. Por eso un `THROW` bien redactado es la manera en que las reglas de la base hablan directamente con el usuario.

13.6 ¿Qué problema resuelven `UPDLOCK` y `HOLDLOCK`? ¿Qué pasaría sin ellos?

Resuelven la concurrencia: dos personas haciendo lo mismo al mismo tiempo. El caso de libro es `sp_RegistrarMovimientoConsumible`. Hay 10 resmas de papel. Dos encargados registran a la vez una salida de 8. Sin protección ocurre esto: el primero lee 10, el segundo también lee 10, ambos comprueban que 8 ⩽ 10 y ambos escriben su resultado; el stock termina en 2 y se entregaron 16 resmas de las 10 que había. A esto se le llama actualización perdida.

El procedimiento lo evita leyendo el stock con `WITH (UPDLOCK, HOLDLOCK)` dentro de la transacción. `UPDLOCK` avisa «leo esto porque lo voy a modificar», de modo que ningún otro pueda tomar el mismo dato con la misma intención; `HOLDLOCK` mantiene ese bloqueo hasta que la transacción termine, no solo mientras dura la lectura. El resultado: el segundo encargado queda esperando en esa línea. Cuando el primero confirma, el segundo lee el stock real (2) y la comprobación `@Cantidad > @Stock` lo rechaza con el error 50043. La hora de llegada decide quién gana, y nadie ve datos falsos.

Se usa el mismo patrón en `sp_RegistrarDevolucion`, que bloquea con `UPDLOCK` la fila del préstamo antes de comprobar que sigue activo: si alguien pulsa dos veces «Devolver» o dos equipos la registran a la vez, la segunda llamada encuentra el préstamo ya devuelto y responde 50037. Y en `sp_GuardarSalon`, para calcular el siguiente número de salón.

No siempre se usa un bloqueo, y es importante entender la alternativa. `sp_RegistrarPrestamo` revisa que la unidad no esté prestada con un `SELECT` normal; si dos personas lo hicieran en el mismo instante, ambas pasarían la revisión. Lo que lo impide es el índice único filtrado `UX_Prestamos_UnidadActiva`: el segundo `INSERT` sería rechazado por la propia base, aunque con un mensaje técnico en lugar del amable. La integridad nunca está en riesgo; solo cambia la calidad del mensaje en una coincidencia muy poco probable.

13.7 ¿Cómo evitan la inyección SQL?

La inyección ocurre cuando el programa construye una consulta pegando texto del usuario dentro del SQL. Si el código fuera `"SELECT * FROM Empleados WHERE Nombre = '" + textBox.Text + "'"` y alguien escribiera `'; DROP TABLE Empleados; --`, el motor

recibiría una orden completa que incluye un `DROP`. El dato se convirtió en código.

Casi todo el acceso a datos pasa por los métodos de la clase `Datos`, y todos reciben los valores como parámetros: `Datos.Tabla("SELECT ... WHERE IdEmpleado = @id", Datos.P("@id", id))`. Las pocas pantallas que arman su propio `SqlCommand`, como el login, hacen lo mismo con `cmd.Parameters.AddWithValue("@user", user)`. Un parámetro viaja al servidor separado del texto de la consulta, y el motor lo trata siempre como un valor, nunca como una instrucción. Aunque alguien escriba el texto más malicioso en el campo de usuario, se buscará literalmente como un nombre extraño y no encontrará nada. `Datos.PTexto` además convierte el texto vacío en `NULL`, para no guardar cadenas en blanco. En algunas pantallas verás también filtros como `"Tipo = '" + texto + "'"`; no son consultas al servidor sino filtros en memoria (`DataView.RowFilter`) sobre una tabla que ya se cargó, así que no pueden alterar la base.

Los procedimientos refuerzan la idea. Un procedimiento recibe tipos concretos (`@IdUnidad INT`, `@FechaLimite DATETIME`): si llega algo que no cabe en el tipo, falla antes de ejecutarse. Y todas las listas, como los identificadores de varias unidades, se procesan con `STRING_SPLIT` convirtiendo cada pieza a número con `CAST(value AS INT)`, de modo que un texto no numérico no se ejecuta, simplemente da error.

Además, por seguridad adicional, las contraseñas no se comparan en SQL: se guardan como hash BCrypt y la verificación se hace en C#. Eso no es inyección, pero es parte de la misma idea: lo sensible nunca viaja ni se guarda en claro.

13.8 ¿Cómo recibe `sp_CambiarEstadoUnidades` una lista de unidades y por qué lo hace así?

La pantalla de Unidades permite seleccionar varias y cambiarles el estado juntas. El procedimiento recibe esa lista como un solo texto: `@Ids VARCHAR(MAX)`, por ejemplo `'12,15,18'`. Con `STRING_SPLIT(@Ids, ',')` lo convierte en una tabla de filas, y con `SELECT DISTINCT CAST(value AS INT)` obtiene números sin repetir, que guarda en una variable de tabla `@sel`. Esa pequeña tabla temporal es la «selección», y todas las comprobaciones posteriores se hacen contra ella con `JOIN` y `EXISTS`.

Las validaciones se encadenan en un orden con sentido. Primero, que el estado exista (50910). Segundo, que todas las unidades existan y pertenezcan al activo indicado (50911), lo cual evita que alguien mande ids ajenos. Tercero, que ninguna esté prestada si se quiere dar de baja (50912). Cuarto, que no se marque como «Bueno» una unidad con un reporte

de daño abierto (50913). Quinto, que no se intente sacar de baja una unidad dada de baja (50914). Y sexto, que la baja traiga un motivo de al menos cinco caracteres (50962). Si cualquiera falla, el `THROW` corta antes de tocar nada.

La razón de enviarlas todas en una llamada, en lugar de que C# haga un bucle y llame una vez por unidad, es doble. Es más rápido: un viaje a la base en vez de veinte. Y es más seguro: un único `UPDATE` es atómico, así que no puede quedar la mitad de las unidades cambiada si algo falla a la mitad. Es lo que se llama un procesamiento por conjuntos: pensar en «todas a la vez» en lugar de «una por una».

13.9 La regla de la baja aparece repetida en cuatro procedimientos. ¿No es un riesgo?

Es una duplicación deliberada, y vale la pena saber defenderla y también reconocer su costo. La regla de dar de baja una unidad tiene cuatro puntos: no puede estar prestada (50912), el motivo debe tener al menos cinco caracteres (50962), una baja no se revierte (50914), y la fecha y el motivo se escriben solo cuando la unidad *entra* en baja, no cada vez que se toque. Hay cuatro caminos legítimos que pueden llevar a una baja: `sp_CambiarEstadoUnidades` (varias a la vez), `sp_CambiarEstadoUnidad` (una sola), `sp_RegistrarMantenimiento` (el motivo es la descripción del servicio) y `sp_RegistrarDevolucion` (el motivo es la observación de la devolución).

Si la regla estuviera solo en uno, los otros tres serían puertas traseras. Por eso se aplicó en los cuatro, y cada comentario lo dice con una frase como «las mismas de sp_CambiarEstadoUnidades», para que quien modifique una sepa que tiene que revisar las demás. En `sp_RegistrarDevolucion` falta la comprobación de «no prestada» a propósito: ahí el préstamo es precisamente lo que se está cerrando.

El costo es de mantenimiento: si mañana cambia el mínimo de cinco caracteres a diez, hay que acordarse de cambiarlo en cuatro lugares. La alternativa más limpia sería concentrar las reglas en un único procedimiento auxiliar que los demás llamen, y es una mejora razonable. Si te lo preguntan, puedes decir que se priorizó que ningún camino se saltara la regla, y que centralizarla es el siguiente paso natural.

Otro detalle fino: el `CASE` que escribe `FechaBaja` y `MotivoBaja` tiene tres ramas. Si el nuevo estado no es baja, no toca nada; si ya estaba de baja, conserva los datos originales y solo completa los que faltaban; y solo si la unidad *entra* en baja, escribe la fecha de hoy y el motivo. Así el registro de una baja no se sobrescribe por accidente.

13.10 ¿Cómo funciona `sp_GenerarPlanilla`, y para qué sirve el parámetro `@Simular`?

Generar la planilla del mes significa crear un pago pendiente para cada empleado que corresponda. El procedimiento empieza decidiendo quiénes son los candidatos: empleados que no estén dados de baja y que ya estuvieran contratados al final de ese mes. Los guarda en una tabla temporal `#cand` junto con su salario base, bono y descuento (que vienen de su estructura salarial en `Planillas`) y una marca `YaTiene`, que vale 1 si ya existe un pago vigente de ese mes.

Sobre esa lista calcula tres cifras: cuántos pagos se generarán, cuántos ya existían y cuántos empleados quedan sin generar porque su estructura no tiene salario, de los que además arma la lista de nombres con `STRING_AGG`. Aquí entra `@Simular`. Con `@Simular = 1` el procedimiento devuelve esas cifras y termina con `RETURN` sin escribir nada. Con `@Simular = 0` hace el trabajo real. La pantalla `frmGenerarPlanilla` lo usa en dos tiempos: cada vez que eliges un mes llama en modo simulación y muestra algo como «Se generarán 12 pagos, ya tenían pago 3, sin salario asignado 2: Fulano, Mengano», y si el mes está cerrado o no hay nada que generar, deshabilita el botón. Solo cuando pulsas Generar y confirmas, llama otra vez con `@Simular = 0`. La vista previa evita sorpresas y permite corregir antes de generar.

La generación real es un único `INSERT ... SELECT` dentro de una transacción. Por eso es seguro: o se crean todos los pagos o no se crea ninguno. También es repetible: gracias a `YaTiene`, ejecutarlo dos veces el mismo mes no duplica pagos (y el índice único filtrado de `PagosPlanilla` sería la defensa de respaldo). Antes de escribir rechaza los meses futuros (50802) y los cerrados (50800). El descuento de cada pago es el descuento de ley de su estructura más el descuento por ausencias del mes, limitado con un `CASE` para que nunca supere el salario base más el bono, y así no violar el `CHECK` de montos.

13.11 ¿Qué pasa al cerrar y al reabrir un mes?

Cerrar un mes es declararlo definitivo: desde ese momento no se pueden generar, editar, marcar como pagados ni anular pagos de ese mes, ni registrar ausencias que caigan en él. `sp_CerrarPeriodo` es muy cuidadoso antes de escribir. Verifica en orden que el mes no sea futuro (50808), que no esté ya cerrado (50804), que tenga al menos un pago (50805) y que no queden pagos pendientes (50803): el mensaje incluso dice cuántos son y qué hacer, «márquelos como pagados o anúlelos». Esta última regla es de negocio puro: no se cierra un mes con deudas sin resolver.

Solo entonces abre la transacción, que tiene dos escrituras: marca el periodo como cerrado en `PeriodosPlanilla` (si la fila no existía la crea, y si existía la actualiza) y anota la acción en `PeriodosPlanillaHistorial`, con el usuario y la fecha.

Reabrir exige más, porque deshace algo que se había declarado definitivo. `sp_ReabrirPeriodo` comprueba que el mes realmente esté cerrado (50806) y exige un motivo de al menos cinco caracteres (50807), que queda guardado en el historial junto con quién lo reabrió. Además, el cierre y la reapertura quedan en la auditoría general, porque `PeriodosPlanilla` tiene su trigger, y `vw_Auditoria` los traduce a «Cerró el mes» y «Reabrió el mes». Todo cambio sobre un mes cerrado deja huella. Esa trazabilidad es lo que le da valor al cierre.

13.12 En `sp_GuardarConfiguracionInicial`, ¿por qué la configuración y el administrador se guardan dentro de la misma transacción?

Porque si no, el sistema podría quedar atrapado. Fíjate en el orden del procedimiento. Lo primero que comprueba es que no exista ya una configuración (si existe, lanza 50021), y esa comprobación es justamente lo que cierra el asistente de primer uso para siempre. Después, dentro de la transacción, inserta la fila de `ConfiguracionEmpresa` y, a continuación, crea el usuario administrador.

Supón que se guardara la configuración y luego fallara la creación del administrador, por ejemplo porque el nombre de usuario ya existe (50022) o porque no existe el rol Administrador (50023). Sin transacción, la configuración ya estaría guardada; al abrir el programa otra vez, el sistema creería estar configurado, el asistente no volvería a aparecer, y no habría ningún usuario con quien entrar. Una instalación inutilizable, sin forma de repararla desde el programa. Con la transacción, cualquier fallo deshace *también* la configuración, y el sistema vuelve al estado «sin configurar» para que se pueda intentar de nuevo.

Es una gran lección sobre el propósito de las transacciones: no solo se usan para sumar y restar, sino para proteger estados que no pueden quedar a medias. También conviene mencionar que la contraseña llega ya convertida en hash BCrypt desde C#, así que el procedimiento nunca ve ni guarda la clave real.

13.13 Si un procedimiento lanza un error, ¿cómo se ve eso en la pantalla? Sigue un caso de principio a fin

Tomemos la salida de 50 hojas cuando solo quedan 20. La persona escribe la cantidad en la pantalla de Consumibles y pulsa Registrar. El programa reúne el consumible, el tipo, la cantidad, el motivo y el usuario, y llama a `sp_RegistrarMovimientoConsumible` con `Datos.Procedimiento`. Dentro del procedimiento, la validación del tipo y de la cantidad pasa; se abre la transacción; se lee el stock con bloqueo y vale 20; la comprobación `@Cantidad > @Stock` es verdadera y se ejecuta `THROW 50043`. Como el `THROW` está dentro del `TRY`, el control salta al `CATCH`, que ve que hay una transacción abierta y la deshace con `ROLLBACK`. Después ejecuta el `THROW;` sin parámetros, que vuelve a lanzar el error original hacia afuera.

En C#, ese error llega como una `SqlException`. El código de la pantalla la atrapa con `catch (SqlException ex)` y muestra `ex.Message` en un cuadro de mensaje: «Stock insuficiente: no se puede despachar más de lo que hay». La persona entiende qué pasó y qué hacer, y ni el stock ni el kardex cambiaron.

Hay además otras dos capas. La pantalla ya intenta evitar que se llegue ahí: muestra cuánto hay disponible y limita el número que se puede escribir, así que lo normal es que la persona nunca provoque el error. Y para los errores que no son de los procedimientos, como una llave duplicada o una llave foránea violada, C# revisa el número del error (`ex.Number`): 2601 y 2627 son duplicados, 547 es una restricción de llave foránea o `CHECK`. Esa combinación completa el cuadro: validación amable en la pantalla, regla clara con mensaje propio en el procedimiento, y restricciones de la tabla como última defensa.

14. Preguntas y respuestas, bloque 5: triggers, auditoría y seguridad

Este bloque cubre dos de los indicadores de la rúbrica: el manejo de la seguridad (contraseñas, recuperación, trazabilidad) y la solidez de la base. Las respuestas parten del código real, así que cuando una dice «el sistema hace X» puedes abrir el script o la pantalla y mostrarlo.

14.1 ¿Qué es un trigger y en qué momento se ejecuta?

Un trigger (disparador) es código de la base de datos que se ejecuta solo, sin que nadie lo llame, cuando ocurre un cambio en una tabla. Con un procedimiento tú decides cuándo correr (`EXEC ...`); con un trigger, es el propio `INSERT`, `UPDATE` o `DELETE` el que lo

despierta. Por eso es la herramienta ideal para lo que debe pasar siempre, sin excepción y sin depender de que alguien se acuerde: dejar constancia de un cambio.

Tus triggers de auditoría son de tipo `AFTER INSERT, UPDATE, DELETE`: se ejecutan después de que el cambio se aplicó, pero dentro de la misma operación. Eso tiene dos consecuencias importantes. La primera es que forman parte del mismo trabajo que los disparó: si el trigger falla, el cambio completo se cancela, de modo que no puede existir un cambio sin su registro. La segunda es que un trigger se ejecuta una vez por sentencia, no una vez por fila. Si un `UPDATE` modifica veinte empleados, el trigger corre una sola vez y recibe las veinte filas juntas. Por eso se escribe pensando en conjuntos y no en filas sueltas, y por eso el de auditoría inserta un registro por cada fila afectada en una sola instrucción.

Existe otro tipo, `INSTEAD OF`, que significa «en lugar de». En ese caso el cambio original no se ejecuta: el trigger lo reemplaza. Lo usa `trg_Auditoria_Inmutable`, que se explica en la pregunta 14.6.

14.2 ¿Qué son las tablas `inserted` y `deleted` y cómo sabe el trigger si fue alta, cambio o baja?

Dentro de un trigger existen dos tablas temporales que el motor llena automáticamente con lo que acaba de pasar. `inserted` contiene las filas nuevas, tal como quedaron. `deleted` contiene las filas viejas, tal como estaban antes. Según la operación: en un `INSERT` solo hay filas en `inserted`; en un `DELETE` solo hay filas en `deleted`; y en un `UPDATE` hay filas en las dos, porque modificar es, para el motor, quitar la versión vieja y poner la nueva.

Con eso el trigger deduce qué ocurrió con una pregunta simple: ¿hay filas en las dos? Entonces es un `UPDATE` (`U`). ¿Solo en `inserted`? Un alta (`I`). ¿Solo en `deleted`? Una baja (`D`). Eso es lo que hace el `CASE` del trigger genérico.

Para emparejar la fila vieja con la nueva en un `UPDATE`, el trigger une las dos tablas por la llave primaria con un `FULL OUTER JOIN`. Se usa *full* porque en un alta no hay pareja en `deleted` y en una baja no hay pareja en `inserted`; un `JOIN` normal haría desaparecer justo esos casos. Ya con cada pareja, el trigger convierte la fila vieja y la nueva a JSON y las guarda en `Antes` y `Despues`. Fíjate también en el filtro final: `WHERE @op <> 'U' OR Antes <> Despues`. Si un `UPDATE` no cambió realmente ningún valor, por ejemplo cuando se guarda un formulario sin tocar nada, no se registra nada. Así el historial muestra cambios reales y no ruido.

14.3 ¿Cómo sabe el trigger *quién* hizo el cambio, si la conexión siempre es del mismo equipo?

Es una buena pregunta porque el programa se conecta a la base con la autenticación de Windows (`Integrated Security`), de manera que, para SQL Server, el «usuario» siempre es la cuenta de Windows de la computadora, no «María» ni «Carlos» que iniciaron sesión en tu programa. Si el trigger solo pudiera ver la cuenta de Windows, todo el historial diría lo mismo.

La solución usa `SESSION_CONTEXT` , un espacio de memoria asociado a la conexión donde se pueden guardar pares clave-valor. Cuando el programa abre una conexión, `Conexion.Conectar()` engancha un evento (`StateChange`) y, en el instante en que la conexión pasa de cerrada a abierta, ejecuta `sp_set_session_context @key = 'Usuario', @value = <el usuario que inició sesión en el programa>` . Desde ese momento, cualquier trigger que se dispare en esa conexión puede leer `SESSION_CONTEXT(N'Usuario')` y sabe quién está actuando. Es el equivalente a ponerle una etiqueta con el nombre a cada conexión.

Si no hay etiqueta, por ejemplo cuando alguien modifica datos directamente desde Management Studio, el trigger genérico guarda `NULL` y la vista `vw_Auditoria` lo muestra como `(sistema)` . Los triggers especiales de inventario (`trg_AuditoriaUnidades` , `trg_AuditoriaActivos`) usan un respaldo: `ISNULL(SESSION_CONTEXT(...), SUSER_SNAME())` , es decir, si no hay usuario del programa, guardan la cuenta de Windows. También se guarda en cada fila el nombre del equipo (`Equipo` , con el valor por defecto `HOST_NAME()`), que ayuda a saber desde qué computadora se hizo.

Una limitación honesta: el bloque que pone la etiqueta tiene un `catch` vacío, de modo que si fallara, el programa seguiría funcionando pero esa conexión no tendría usuario y los cambios saldrían como `(sistema)` . Se eligió así para que un fallo de auditoría nunca impida trabajar.

14.4 ¿Por qué un procedimiento *genera* los triggers de auditoría en vez de escribir uno por tabla a mano?

Porque son diecisiete tablas con la misma lógica, y escribir diecisiete triggers casi idénticos a mano es una receta para errores: bastaría olvidar una columna en uno para que su historial sea incompleto. `sp_CrearTriggerAuditoria` recibe el nombre de una tabla y construye su trigger a medida.

Para lograrlo usa SQL dinámico: arma el texto del `CREATE TRIGGER` como una cadena y lo ejecuta con `sp_executesql`. Primero consulta el catálogo del sistema (`sys.indexes`, `sys.columns`) para descubrir cuál es la llave primaria de la tabla, de cuántas columnas es, y qué columnas tiene. Si la llave es compuesta, como en `EmpleadoMaterias`, une las partes con guion para guardar algo como `7-3` como identificador de la fila. Descarta las columnas de tipo binario (fotos, logotipo) porque no tiene sentido guardarlas como JSON en cada cambio, y acepta un parámetro `@Excluir` para dejar fuera columnas sensibles o ruidosas: en `Usuarios` se excluyen `Clave` y `UltimoAcceso`.

Vale la pena señalar que el SQL dinámico es un lugar clásico de riesgo de inyección, y aquí está protegido con `QUOTENAME`, que encierra cada nombre de tabla o columna entre corchetes e impide que un nombre malicioso se salga de su sitio. Además, el procedimiento solo lo ejecuta a mano quien administra la base, con nombres de tabla que él mismo escribe, no con datos de usuarios finales.

El resultado es doble. Los diecisiete triggers son idénticos en comportamiento por construcción. Y si mañana se agrega una tabla nueva o una columna nueva, basta volver a ejecutar el generador para regenerar el trigger con la columna incluida. Para que se pudieran leer, el script incluye los triggers ya generados completos, y una frase en un comentario explica cómo se produjeron.

14.5 ¿Qué se guarda en la tabla `Auditoria` y cómo se ve un cambio?

Cada fila de `Auditoria` responde a seis preguntas: cuándo (`FechaHora`, con la hora del servidor), quién (`Usuario`), desde dónde (`Equipo`), en qué tabla (`Tabla`), qué se hizo (`Operacion`: `I` alta, `U` modificación, `D` baja, `A` acceso al sistema) y sobre qué fila (`Clave`, el identificador). Y dos columnas de texto grande, `Antes` y `Despues`, con el contenido de la fila en formato JSON. La operación está restringida por un `CHECK` a esas cuatro letras.

Un ejemplo: se sube el salario de un pago de 600 a 650. La fila de auditoría tendría `Tabla = 'PagosPlanilla'`, `Operacion = 'U'`, `Clave = '41'`, `Antes = {"IdPago":41, "SalarioBase":600.00, "Bono":0.00, ...}` y `Despues = {"IdPago":41, "SalarioBase":650.00, "Bono":0.00, ...}`. El JSON guarda *toda* la fila, no solo lo que cambió. Es la vista `vw_Auditoria`, con la función `fn_AuditoriaCambios`, la que compara los dos textos y muestra solo la diferencia, `SalarioBase: 600.00 → 650.00`.

Guardar la fila completa antes y después es una decisión de diseño con ventajas claras. Permite reconstruir cómo era un dato en cualquier momento. No depende de las columnas de cada tabla. Y deja evidencia suficiente si alguien pregunta «¿qué había aquí antes?». El

costo es espacio en disco, pero para el volumen de una escuela es insignificante, y por eso la tabla tiene tres índices (por fecha, por usuario y por tabla y clave) para que la consulta de la pantalla de Registro de actividad siga siendo rápida cuando crezca.

14.6 ¿Por qué el registro de actividad no se puede borrar ni editar? ¿Qué tan seguro es?

Porque una auditoría que se puede modificar no sirve como prueba. Si quien hizo un cambio indebido pudiera borrar su rastro, el historial perdería todo su valor. `trg_Auditoria_Inmutable` es un trigger `INSTEAD OF UPDATE, DELETE` sobre la tabla `Auditoria`: cuando alguien intenta modificar o borrar una fila, el trigger se ejecuta *en lugar de* la operación y lanza el error 50900, «El registro de actividad no se puede modificar ni borrar». La operación original nunca ocurre. Las altas sí están permitidas, y deben estarlo: es la forma en que se escribe el historial.

Hay que distinguir qué amenazas cubre. Protege contra errores y contra uso indebido desde el programa o desde consultas normales. No es una defensa absoluta: una persona con permisos de administrador de la base de datos podría desactivar el trigger, borrar la tabla o editar el archivo de la base. Para una protección total se necesitarían otras cosas, como permisos que impidan a todos (incluidos los administradores del programa) tocar la estructura, copias fuera del equipo, o las tablas de libro mayor («ledger») que ofrecen versiones modernas de SQL Server. Lo honesto es presentarlo como una barrera real y razonable para el alcance del sistema, no como invulnerable. Y eso mismo demuestra que conoces el tema.

14.7 Si ya hay triggers genéricos, ¿por qué existen además `trg_AuditoriaUnidades` y `trg_AuditoriaActivos`?

Porque sirven a fines distintos aunque vigilen las mismas tablas. El trigger genérico guarda el JSON completo en `Auditoria`: es un registro universal pensado para responder «¿qué cambió y quién fue?» en cualquier módulo, y se muestra en la pantalla Registro de actividad. Los especiales alimentan las tablas `AuditoriaUnidades` y `AuditoriaActivos`, que tienen columnas propias y estructuradas: estado anterior y nuevo, salón anterior y nuevo, nombre anterior y nuevo. Esa estructura permite construir una línea de tiempo legible de cada equipo («pasó de Bueno a En reparación, del aula 3 a la bodega») sin tener que interpretar JSON.

También son más selectivos. `trg_AuditoriaUnidades` registra una modificación solo si cambió el estado o el salón; si se corrige una observación, no deja huella en esa línea de tiempo (aunque el genérico sí la guarda). `trg_AuditoriaActivos` solo anota altas, bajas y

cambios de nombre. Cada uno tiene tres bloques `INSERT`, uno por operación, y cada bloque usa `NOT EXISTS (SELECT 1 FROM deleted)` o su equivalente para saber de qué operación se trata.

Si te preguntan si no es redundante, la respuesta es que hay dos niveles de detalle para dos públicos. El registro de actividad es para quien administra y audita todo el sistema. El historial de cada equipo es para quien consulta la vida de una pieza. Los dos coexisten sin estorbarse porque son tablas distintas.

14.8 ¿Qué se registra de los accesos al sistema y por qué importa?

Además de los cambios en las tablas, el sistema deja constancia de quién entra. Al iniciar sesión, al cerrar sesión y en cada intento rechazado, el programa llama a `sp_RegistrarAcceso`, que inserta en `Auditoria` una fila con `Tabla = 'Acceso'` y `Operacion = 'A'`, con el resultado dentro del JSON. Los resultados posibles son «Inició sesión», «Cerró sesión», «Intento fallido», «Intento con cuenta desactivada» e «Intento sin permiso», de modo que queda claro *por qué* se rechazó cada intento. En un inicio de sesión exitoso, además actualiza `Usuarios.UltimoAcceso`. La pantalla de actividad los muestra como «Acceso al sistema», con el nombre del equipo.

Importa por dos razones. La primera es que los intentos fallidos son la señal de alguien que intenta adivinar una contraseña: tenerlos registrados permite detectarlo después. La segunda es de responsabilidad: si ocurre algo extraño con los datos, se puede cruzar la hora del cambio con quién estaba dentro del sistema.

Hay un detalle fino que conecta con lo anterior. El trigger de `Usuarios` excluye la columna `UltimoAcceso`. Si no lo hiciera, cada inicio de sesión actualizaría esa columna y generaría *otro* registro de auditoría por el solo hecho de entrar, duplicando el ruido. Por la misma lógica excluye `Clave`: el historial nunca guarda contraseñas, ni siquiera en forma de hash.

14.9 ¿Cómo se protegen las contraseñas?

Nunca se guardan tal cual. En la columna `Usuarios.Clave` hay un hash BCrypt, algo como `$2a$11$...`. Un hash es el resultado de pasar la contraseña por una función matemática de un solo sentido: dado el hash, no se puede recuperar la contraseña. BCrypt tiene dos características que lo hacen apropiado para contraseñas. Incluye una *sal* aleatoria propia de cada hash, de modo que dos usuarios con la misma contraseña tienen hashes distintos y no se pueden usar tablas precalculadas. Y es deliberadamente lento (tiene un «costo» ajustable), así que probar millones de combinaciones por fuerza bruta se vuelve muy caro para un atacante.

Para comprobar una contraseña, el login lee el hash de la base con una consulta parametrizada y llama a `PasswordHasher.Verificar` en C#, que vuelve a calcular y compara. La base de datos jamás ve la contraseña en claro: llega ya convertida en hash a `sp_CrearUsuario`, `sp_CambiarMiClave` y `sp_GuardarConfiguracionInicial`.

Existe un detalle de compatibilidad que conviene conocer. El login acepta una clave antigua guardada en texto plano (reconocible porque no empieza por `$2`), pero solo para migrarla: si coincide, la reemplaza de inmediato por su hash BCrypt, y a partir de ahí ya no existe en claro. Es una forma de actualizar datos viejos sin obligar a nadie a hacer nada.

Otras dos medidas se apoyan en la columna `DebeCambiarClave`: cuando un administrador crea otro usuario o le restablece la contraseña, esta queda como temporal y el sistema obliga a cambiarla en el primer ingreso. Así quien creó la cuenta no conoce la contraseña definitiva de la otra persona.

14.10 Explica el flujo de recuperación de contraseña por correo, paso a paso

En la pantalla de login, «¿Olvidó su contraseña?» abre el formulario de recuperación. Primero, la persona escribe su correo y el programa comprueba que ese correo exista en `Usuarios`; si no, avisa. Después genera un código de seis dígitos con `RandomNumberGenerator`, el generador criptográfico del sistema, y no con `Random`, cuyos números son predecibles y no sirven para seguridad. Lo envía por correo mediante el servicio de correo del sistema, y lo guarda solo en la memoria del formulario junto con la hora de vencimiento, quince minutos después. No se escribe en la base de datos.

Después de enviarlo, el formulario habilita el segundo paso: la persona escribe el código recibido, la contraseña nueva y su confirmación. Al pulsar Restablecer, el programa revisa en este orden que la contraseña tenga al menos seis caracteres, que las dos coincidan, que el código no haya vencido y que el código sea el correcto. Si es incorrecto suma un intento fallido; al llegar a cinco, el código se invalida y hay que pedir uno nuevo. Esto frena la adivinación: el código puede ser cualquiera de unos novecientos mil números, y con cinco intentos la probabilidad de acertar es de aproximadamente uno entre ciento ochenta mil. Si todo está bien, la contraseña nueva se convierte en hash BCrypt y se guarda con `UPDATE Usuarios SET Clave = @clave WHERE Correo = @correo`.

Algunas observaciones que te harán sonar preparado. La contraseña nueva se guarda como hash, igual que cualquier otra. La actualización va directamente en C# con parámetros, no por un procedimiento almacenado, y es una de las pocas operaciones que quedaron así. Que el correo sea único por usuario lo garantizan los procedimientos de creación y edición (errores 50904 y 50024), no un índice único en la tabla; sería una

mejora blindarlo a nivel de base. Y el mensaje «este correo no está registrado» le dice a quien lo intenta si un correo existe o no, lo cual es una compensación habitual entre claridad para el usuario y discreción. Cualquiera de esas observaciones, planteada como mejora y no como defecto, demuestra criterio.

14.11 ¿Cómo evitan que el sistema se quede sin administradores o que alguien se desactive a sí mismo?

`sp_CambiarEstadoUsuario` aplica dos protecciones antes de desactivar una cuenta. La primera: nadie puede desactivarse a sí mismo (error 50905), porque dejaría la sesión actual en un estado absurdo. La segunda: debe quedar al menos otro administrador activo (error 50906), comprobado con un `NOT EXISTS` que busca otro administrador activo distinto del que se desactiva.

Lo más interesante es cómo lo protege contra la concurrencia. Piensa en dos administradores, A y B, y nadie más. A desactiva a B y, al mismo instante, B desactiva a A. Cada procedimiento revisa que quede otro administrador activo; cada uno ve que sí (el otro todavía figura activo), y ambos proceden. Resultado: cero administradores activos y nadie puede entrar, un desastre que ninguna validación de pantalla detectaría. El procedimiento lo evita leyendo el usuario con `WITH (UPDLOCK, HOLDLOCK)` dentro de una transacción: la segunda llamada espera a que la primera confirme, y cuando se ejecuta ya ve el estado real, en el que solo queda un administrador, y se rechaza con el 50906.

Es un ejemplo perfecto de cómo se combinan tres ideas de este documento: una regla de negocio en un procedimiento, una transacción y un bloqueo. También demuestra que la integridad no depende de que los usuarios sean cuidadosos.

14.12 ¿Qué protecciones tiene el login contra quien intenta adivinar contraseñas? ¿Y quién puede entrar?

Hay varias capas pequeñas que suman. Después de cinco intentos fallidos con el mismo usuario, el login se bloquea durante sesenta segundos y muestra cuánto falta; desde el tercer fallo avisa cuántos intentos quedan. Un acierto borra el conteo. Cada intento rechazado queda registrado en la auditoría con su causa (contraseña errónea, cuenta desactivada o falta de permiso; ver 14.8). Las cuentas desactivadas (`Activo = 0`) no pueden entrar aunque tengan la contraseña correcta. Y como la contraseña se verifica con BCrypt, la fuerza bruta sigue siendo cara incluso si alguien llegara a copiar la base de datos.

Sobre quién entra: aunque hay dos roles en la tabla `Roles` (`Administrador` y `Empleado`), el programa solo deja iniciar sesión a los administradores. La tabla de roles existe para que el modelo pueda crecer sin rediseñarse, por ejemplo con un rol de consulta que solo lea datos.

Como siempre, vale mencionar los límites con naturalidad. El conteo de fallos y el bloqueo viven en la memoria del programa: si se cierra y se vuelve a abrir, el contador se reinicia, y es por usuario, no por equipo. Está pensado para frenar el uso casual de fuerza bruta en una instalación local, no para resistir un ataque determinado, que requeriría un bloqueo guardado en la base. Y es coherente con el contexto del sistema, que corre en una sola computadora de la institución.

15. Preguntas y respuestas, bloque 6: integración con el sistema y preguntas trampa

Este último bloque junta las piezas y entra en el terreno incómodo: las preguntas que buscan una debilidad. Una defensa no se gana ocultando los límites del diseño, sino mostrando que los conoces y que sabes por qué están ahí. Por eso varias respuestas incluyen una parte de «qué mejoraría».

15.1 Sigue un préstamo de principio a fin: ¿por qué capas pasa?

La persona abre Gestión de activos, entra a Préstamos, elige una unidad, el tipo de responsable, la persona, el salón de destino y la fecha límite, y pulsa Registrar. En la pantalla ya hay una primera capa de ayuda: los campos obligatorios vacíos muestran un aviso junto al campo, y los selectores solo ofrecen unidades y empleados válidos. Esa capa es comodidad.

Luego `frmGestionActivosPrestamos` llama a `Datos.Procedimiento("sp_RegistrarPrestamo", ...)` pasando cada dato como parámetro. Aquí entra la segunda capa, la que garantiza: el procedimiento comprueba que, si el responsable es un docente o personal administrativo, sea un empleado real y activo con el cargo adecuado (errores 50950 a 50953); que el responsable no esté vacío (50031); que la fecha límite sea futura (50032); que la unidad exista (50033); que no esté en mal estado, en reparación o dada de baja (50034); y que no esté ya prestada (50035). Si pasa todo, hace un solo `INSERT` en `Prestamos`.

La tercera capa es la propia tabla. El `INSERT` debe cumplir los `CHECK` (estado válido, cantidad mayor que cero, tipo de responsable válido), las llaves foráneas (la unidad, el activo, el salón y el empleado deben existir) y el índice único filtrado que impide dos préstamos activos de la misma unidad.

Cuando el `INSERT` termina, el trigger `trg_Aud_Prestamos` se despierta solo y escribe en `Auditoria` quién registró el préstamo, cuándo, desde qué equipo y cómo quedó la fila en JSON. La persona que registró no hizo nada para eso. Finalmente la pantalla recarga la lista desde `vw_Prestamos`, que une el préstamo con el nombre del equipo y el salón, y calcula la `Alerta` con `GETDATE()`. Si el préstamo vence en menos de dos días, la fila aparece en ámbar sin que nadie lo haya programado.

Si en cualquiera de esos pasos algo falla, el error sube por la misma ruta en sentido contrario: la base lanza `THROW` o una restricción, C# recibe una `SqlException`, y la pantalla muestra el mensaje. Poder contar este recorrido sin mirar apuntes demuestra que entiendes cómo conviven las capas.

15.2 ¿Cómo cubre el sistema crear, leer, actualizar y eliminar? ¿En qué se diferencia la eliminación lógica de la directa?

Cada módulo cubre las cuatro operaciones, aunque con distintos caminos. Tomemos Personal. Para crear y editar un empleado, `frmAgregarPersonal` ejecuta un `INSERT` o un `UPDATE` con parámetros. Para leer, `frmGestionPersonal` arma una consulta con `JOIN` a cargos, estados y estructura salarial y la muestra en una tabla con búsqueda y filtros. Para eliminar, no se borra: se hace una **baja lógica**, `UPDATE Empleados SET IdEstado = 2`, y el empleado deja de aparecer en la lista pero conserva su historial. Antes de hacerlo, la pantalla revisa si tiene asuntos pendientes y advierte.

En los módulos con más reglas, como préstamos, consumibles, planillas y ausencias, crear y modificar pasan por procedimientos almacenados, que ya traen las validaciones. La inmensa mayoría de los 34 procedimientos se llaman desde alguna pantalla. Y la eliminación es casi siempre lógica con una bandera: se anula un pago (`sp_AnularPago`), se anula una ausencia (`sp_AnularAusencia`), se da de baja una unidad (estado `Dado de baja`, con fecha y motivo), se desactiva un usuario o un salón.

También hay eliminaciones directas, y todas son de datos sin historia propia o están protegidas por una regla. Desde las pantallas se puede quitar la foto de un empleado (`DELETE FROM EmpleadoFotos`), quitar una carga académica asignada por error (`DELETE FROM AsignacionesDocente`), eliminar una estructura salarial que ningún empleado usa, y eliminar un activo del catálogo que no tiene unidades, préstamos ni mantenimientos. Esas

dos últimas tienen la doble defensa de la que hablamos: la pantalla cuenta usos y avisa con un mensaje claro, y la llave foránea es la red de seguridad si algo se le escapa. Desde procedimientos hay otras dos. `sp_QuitarUnidades` borra unidades, pero solo las que no tienen historial de préstamos, mantenimientos ni reportes, y devuelve cuántas quitó y cuántas no pudo quitar por tener historial. `sp_GuardarMateriasEmpleado` sincroniza la lista de materias de un docente: borra las que ya no están en la lista nueva, inserta las que faltan, y se niega (error 50613) a quitar una materia con la que el docente ya tiene grupos asignados.

La diferencia en una frase: la eliminación directa borra la fila y es irreversible, por eso se reserva para lo que no deja rastro importante; la lógica marca la fila como inactiva o anulada y conserva la historia, por eso se usa en todo lo que tiene relaciones con dinero, personas o inventario. Un detalle que suma: la rúbrica pide «directo o lógico», y el sistema muestra ambos con criterio.

15.3 ¿Las búsquedas y filtros se hacen en la base de datos o en el programa?

Se usan las dos formas, y conviene saber distinguirlas. En la mayoría de pantallas el programa trae una tabla completa con una consulta (`Datos.Tabla(...)`), la envuelve en un `DataView` , y los filtros y búsquedas se aplican en memoria con `RowFilter` , una expresión parecida a un `WHERE` : nombre que contenga ciertas letras, cargo igual a tal, estado igual a cual. Cada vez que se escribe en el cuadro de búsqueda, la tabla se actualiza al instante porque no se va a la base de nuevo.

La otra forma es filtrar en la consulta. La pantalla de Préstamos, por ejemplo, arma la cláusula `WHERE Alerta = 'Atrasado'` y se la manda a `vw_Prestamos` , de modo que la base solo devuelve los atrasados.

El filtro en memoria es rápido y cercano al usuario, y para el tamaño de una escuela (cientos o pocos miles de filas) es lo adecuado. Su límite es que primero hay que traer todas las filas: con cientos de miles de registros sería ineficiente y convendría filtrar en el servidor. Como todo en diseño, es un equilibrio entre simplicidad y escala, y el sistema eligió bien para su escala. Si te preguntan por seguridad, esos filtros en memoria no son consultas al servidor, así que aunque lleven texto del usuario no pueden alterar la base.

15.4 ¿Está normalizada la base de datos? ¿Dónde rompe las reglas a propósito?

En lo esencial, sí, y se puede explicar con las tres formas normales usando tus propias tablas. La **primera forma normal** pide que cada celda contenga un solo valor. Por eso un docente no tiene una celda con «Matemáticas, Física»: sus materias son filas de

`EmpleadoMaterias`. La **segunda forma normal** pide que, si la llave es compuesta, cada columna dependa de toda la llave y no de una parte. `EmpleadoMaterias` es el caso extremo: solo tiene las dos columnas de la llave. La **tercera forma normal** pide que las columnas no dependan de otras columnas que no son la llave. Por eso el nombre del cargo vive en `Cargos` y no repetido en cada empleado, y el nombre del estado de una unidad vive en `Estados`: se guarda el número y se traduce al leer.

También hay desviaciones deliberadas, y son la parte que luce más. `PagosPlanilla` copia el salario base, el bono y el descuento de la estructura salarial: es redundante a propósito, para que el historial de un mes no cambie cuando cambie la estructura. En `Prestamos`, `Responsable` guarda el nombre como texto, `CodigoInventario` repite el código de la unidad e `IdActivo` se podría deducir de `IdUnidad`. Es desnormalización controlada: el préstamo conserva lo que valieron esos datos en el momento en que se hizo (y el nombre de un alumno, que no es empleado, no tiene tabla propia). La regla para explicarlo: se normaliza para evitar contradicciones al guardar, y se repite cuando el dato es una fotografía histórica que no debe cambiar.

15.5 Si alguien renombra el cargo «Docente» o un estado de las unidades, ¿se rompe algo?

Es una pregunta trampa muy buena, porque toca una dependencia real del diseño. Varias reglas reconocen las cosas por su nombre en vez de por su número. `sp_AsignarDocente` y `sp_RegistrarPrestamo` identifican a un docente porque su cargo empieza con «Docente» (`LOWER(NombreCargo) LIKE 'docente%'`). Los procedimientos de baja comparan el estado con el texto `'Dado de baja'`, y las vistas listan estados como `'Malo'` o `'En reparación'`. Si alguien cambiara esos textos, la regla dejaría de reconocerlos y fallaría sin avisar.

El sistema ya se defiende de la parte más expuesta. La pantalla de cargos (`frmAsignarCargo`) impide que un cargo que empieza con «Docente» deje de empezar así, y viceversa, con un mensaje que explica que el sistema reconoce a los docentes por ese nombre. En cuanto a los estados de unidades, son un catálogo fijo que ninguna pantalla permite editar, así que en la práctica el riesgo solo existiría con una intervención directa en la base.

Lo honesto es presentarlo como una compensación: usar el nombre hace el código legible («Dado de baja» se entiende al leerlo), pero lo ata a que el texto no cambie. Hay un caso parecido con números: el estado de empleado Inactivo es el 2, y aparece como `IdEstado`

`<> 2` en seis lugares del script. Si te preguntan qué harías, la mejora es dejar de comparar textos y números sueltos y guardar esas identidades en un solo lugar (una columna de código estable en el catálogo, o funciones como `fn_IdBodega` que las descubran).

15.6 ¿Por qué el nombre del empleado es único? ¿Y si hay dos personas que se llaman igual?

`UQ_Empleados_Nombre` obliga a que no existan dos empleados con exactamente el mismo nombre. La intención es evitar duplicados accidentales: que alguien dé de alta dos veces a la misma persona. Es una restricción simple y efectiva para una institución pequeña, donde además las listas desplegables y los reportes se leen mejor si cada nombre es inequívoco.

Su límite es evidente: dos personas distintas con el mismo nombre completo no podrían coexistir, y habría que distinguirlas añadiendo, por ejemplo, un segundo apellido o una inicial. El identificador verdadero de una persona no es su nombre sino su DUI, y esa es la razón por la que existe `UX_Empleados_DUI`, único y opcional. La mejora lógica sería confiar en el DUI (que no se repite entre personas) para la unicidad real, y dejar el nombre sin restricción. Planteado así, la pregunta se convierte en una oportunidad de mostrar que distingues entre un identificador natural y un nombre que podría repetirse.

15.7 ¿Qué pasa si se va la luz justo mientras se genera la planilla?

La base se queda como estaba antes de empezar, sin pagos a medias. Esto lo garantizan dos cosas que trabajan juntas. La primera es que la generación es un solo `INSERT ... SELECT` dentro de una transacción, así que desde el punto de vista lógico es todo o nada. La segunda es la durabilidad que ofrece SQL Server: antes de hacer cualquier cambio en los archivos de datos, el motor lo escribe en un registro de transacciones (el «log»). Si el equipo se apaga a mitad de camino, al volver a iniciar el motor revisa ese registro, deshace lo que quedó sin confirmar y rehace lo que sí estaba confirmado. A este proceso se le llama recuperación automática.

Las consecuencias para la persona son simples: si le aparecía el mensaje de «planilla generada», los pagos existen; si el equipo se cayó antes, no existe ninguno y puede volver a generar sin riesgo de duplicados, porque el procedimiento salta a quienes ya tienen pago y además hay un índice único filtrado de respaldo. La misma lectura sirve para cualquier procedimiento con transacción del sistema.

15.8 ¿Se puede usar el sistema desde varias computadoras a la vez?

Así como está, está pensado para una sola computadora. La conexión apunta a `(localdb)\MSSQLLocalDB`, una versión ligera de SQL Server que vive en el equipo de quien lo usa y se inicia bajo demanda. Eso es perfecto para desarrollar y presentar, porque no requiere instalar un servidor, pero otra computadora no puede conectarse a ella.

Para usarlo en red habría que instalar una instancia de SQL Server (la edición Express es gratuita) en un equipo que haga de servidor y cambiar la dirección en un solo lugar del código, la constante `Servidor` de la clase `Conexion`. Todas las pantallas obtienen su conexión de ahí (`Conexion.Conectar()`), así que no hay que tocar nada más. Habría que tener en cuenta que el sistema usa autenticación de Windows (`Integrated Security`), que en red exige que las cuentas existan en el servidor; la alternativa sería un usuario propio de SQL Server con su contraseña.

Lo bueno es que la parte difícil del uso concurrente ya está pensada: los bloqueos `UPDLOCK` y `HOLDLOCK`, las transacciones y los índices únicos filtrados existen precisamente para que dos personas trabajando al mismo tiempo no corrompan nada. Es decir, la base está preparada para el uso multiusuario aunque la instalación actual no lo use.

15.9 ¿Cómo se hace una copia de seguridad y cómo se restaura?

Lo más simple es con Management Studio, conectado a `(localdb)\MSSQLLocalDB`: clic derecho sobre la base, Tareas, Copia de seguridad. O con dos instrucciones de T-SQL, que es lo que conviene saber explicar:

```
-- Respaldo (la carpeta debe existir)
BACKUP DATABASE Gestion_administrativa_y_escolar
TO DISK = N'C:\Respaldos\Gestion_administrativa_y_escolar.bak'
WITH INIT, CHECKSUM;

-- Restauración (con el programa cerrado, para que nadie esté usando la base)
RESTORE DATABASE Gestion_administrativa_y_escolar
FROM DISK = N'C:\Respaldos\Gestion_administrativa_y_escolar.bak'
WITH REPLACE;
```

`BACKUP` crea un archivo `.bak` con todo el contenido de la base en ese instante: tablas, vistas, procedimientos, triggers y datos. `INIT` sobrescribe un respaldo anterior con el mismo nombre en lugar de añadir uno nuevo detrás, y `CHECKSUM` verifica que lo escrito no

esté dañado. `RESTORE ... WITH REPLACE` reemplaza la base actual por la del respaldo, y por eso es una operación que se hace con cuidado y con el programa cerrado.

Dos consejos prácticos. Haz una copia justo antes de presentar, con los datos de demostración en el estado en que los quieres mostrar. Y prueba una vez la restauración en tu equipo: un respaldo que nunca se ha restaurado es una promesa, no una garantía. Como la base se puede reconstruir completa con `BaseDeDatos_Completa.sql`, el respaldo sirve sobre todo para conservar los datos, no la estructura.

15.10 Dos detalles prácticos del programa que conviene saber antes de presentar

El primero: al iniciar, el programa decide la primera pantalla con una directiva de compilación. En la configuración **Debug** (la que usa Visual Studio por defecto al pulsar F5), abre directamente el panel principal sin pedir login; en **Release** abre el login. Si presentas en Debug, no podrás mostrar el inicio de sesión, el bloqueo tras fallos ni la recuperación de contraseña. Y como no hay usuario que haya iniciado sesión, los cambios que hagas aparecerán en el registro de actividad a nombre de `(sistema)`, que es exactamente lo que se explicó en la pregunta 14.3. Cambia la configuración a Release en la barra superior de Visual Studio antes de la demostración.

El segundo: el asistente de primer uso aparece solo si la tabla `ConfiguracionEmpresa` está vacía, que es lo que comprueba el programa al arrancar. Si ya lo completaste, no volverá a salir, y además `sp_GuardarConfiguracionInicial` lo impide con el error 50021. Si quieres mostrarlo en vivo, necesitas una base nueva: ejecuta `BaseDeDatos_Completa.sql` sobre otro nombre de base o restaura una copia hecha antes de configurarlo. Lo más cómodo es tener a mano una copia de seguridad «vacía» para esa parte de la demostración.

15.11 ¿Qué mejorarías con más tiempo?

Es la pregunta de cierre más probable, y es mejor responderla con una lista preparada que improvisando. Estas son mejoras reales, ordenadas de menor a mayor esfuerzo, con la razón de cada una.

Primero, un índice único sobre el correo de `Usuarios`. Hoy lo garantizan los procedimientos, pero una restricción de la tabla no se puede esquivar, y la recuperación de contraseña depende de que el correo sea único. Segundo, hacer calculada la columna `SalarioNeto` de `Planillas`, igual que en `PagosPlanilla`, para que sea imposible que se contradiga. Tercero, pasar las operaciones simples que todavía usan `INSERT` y `UPDATE`

directos (cargos, consumibles, empleados) a procedimientos, para tener todas las reglas en un solo lugar. Cuarto, centralizar las reglas de la baja de unidades, hoy repetidas en cuatro procedimientos, en un procedimiento auxiliar.

Quinto, guardar el bloqueo del login en la base en lugar de en la memoria del programa, para que no se reinicie al cerrar la aplicación. Sexto, reemplazar las comparaciones por texto o por números sueltos (cargo «Docente», estado 2, «Dado de baja») por identificadores estables. Séptimo, una tabla de días feriados para que `fn_DiasHabiles` no cuente días festivos como laborables. Y octavo, para uso en red, pasar a una instancia de SQL Server y a un usuario propio de la base. Mencionar tres o cuatro de estas, con su justificación, transmite que el sistema está bien pensado y que su límite es de alcance, no de comprensión.

15.12 Si te piden demostrar que la base se defiende sola, ¿qué haces?

La sección 9.4 de esta guía trae seis consultas listas para ejecutar en Management Studio. Lo que marca la diferencia es narrar qué prueba cada una antes de ejecutarla, para que parezca una demostración y no una lista de comandos.

Con la primera, `vw_Prestamos`, muestras que una vista calcula la alerta sola, sin que nadie la guarde. Con la segunda, `fn_DiasHabiles`, que una función encapsula un cálculo reutilizable (octubre de 2026 tiene 22 días laborables). La tercera, insertar un salón de tipo «Cocina», falla por el `CHECK`, y es la prueba de que la tabla rechaza datos imposibles sin importar quién los mande. La cuarta, intentar modificar `Auditoria`, falla con el 50900 por el trigger `INSTEAD OF`, y prueba que el historial es inalterable. La quinta, pedir más unidades de las que existen, falla con el 50514 desde un procedimiento, y prueba que las reglas de negocio viven en la base. La sexta, `vw_Auditoria`, muestra todo lo anterior ya convertido en frases legibles: quién hizo qué y cuándo.

Un orden narrativo que funciona bien: empiezas por lo que se ve (vista y función), sigues con lo que protege (`CHECK` y procedimiento), sigues con lo que vigila (trigger) y cierras con la prueba de que todo quedó registrado (auditoría). Cuatro minutos, cuatro ideas, y cada una se apoya en algo que ya sabes explicar en profundidad.